

National University of Sciences and Technology

School of Electrical Engineering and Computer Science (SEECS)

Final Year Project Report

For

Adaptive Web Interaction: Leveraging Reinforcement
Learning
for Comprehensive Action Support

Final Year Project

Team Members:

Awais Nazir

Muhammad Saad Ashraf

Muhammad Salman Siddiq

Advisor:

Dr. Faisal Shafait

Date: 2026

Declaration

We hereby declare that this project report entitled “**Adaptive Web Interaction: Leveraging Reinforcement Learning for Comprehensive Action Support**” submitted to the School of Electrical Engineering and Computer Science (SEECS), National University of Sciences and Technology (NUST), is a record of original work done by us under the guidance of our supervisor Dr. Faisal Shafait. No part of this report has been plagiarised without proper citation. This report is submitted in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Software Engineering.

Team Members

Signature

Awais Nazir

Muhammad Saad Ashraf

Muhammad Salman Siddiq

Supervisor

Signature

Dr. Faisal Shafait

Date: _____

Place: Islamabad, Pakistan

Dedication

*To our families, for their patience through the late-night training runs
and the occasional 3 AM checkpoint restart.*

*And to the open-source community—whose open release of models, data,
and tools made this project possible.*

Acknowledgments

We thank our supervisor, Dr. Faisal Shafait, for technical guidance and for the latitude to take this project in the direction the data ultimately demanded—toward the auditing of an existing benchmark rather than the introduction of a new one. We are grateful to the School of Electrical Engineering and Computer Science (SEECs) at the National University of Sciences and Technology for hosting this work and for the computing resources made available during initial exploration.

Outside of NUST, this project would not have been possible without the open release of the InSTA dataset and the `Insta-Qwen3-1.7B-SFT` checkpoint by Trabucco et al., which we used as the starting point for both the audit and the RL training pipeline. We also acknowledge Google’s GenAI free-tier credit programme, which subsidized the Vertex AI Gemini 2.5 Flash judge calls that drove both the feasibility audit and the trajectory evaluation.

Finally, we thank our families and peers for their patience during the more compute-bound stretches of the project—an FYP that depends on a single GPU over several days exposes everyone in proximity to a steady hum and the occasional 3 AM checkpoint-restart.

Abstract

Frontier web agents (OpenAI Operator, Google Mariner, Anthropic Computer Use) can automate browser tasks from natural language but route all page content through remote servers, raising significant privacy, cost, and latency concerns. This project builds **Web Sense**—a privacy-preserving Chrome browser extension that runs a small language model (SLM) entirely on the user’s device via Ollama/GGUF quantization, with no page data leaving the machine, alongside an optional cloud inference mode via OpenRouter. To improve the SLM’s web navigation ability, we fine-tune Qwen3-1.7B with PPO + LoRA reinforcement learning using the InSTA-150k benchmark as a training source. During setup we discover that the benchmark has undergone severe decay: auditing 2,598 sampled tasks with a Gemini-2.5-Flash feasibility judge reveals **only 28.3% of tasks are reliably feasible**—28.6% of target sites are inaccessible, 13.0% reference removed content, and 30.1% are uncertain. We further find that **24.3% of inaccessible sites return HTTP 2xx**, showing that reachability probing alone is insufficient. We clean and filter the dataset down to $\sim 3,000$ doable tasks, of which 600 are sampled for training, and evaluate on a 100-task held-out filtered set. RL training yields **a +17.1 percentage-point improvement in mean judge-success score (from 29.05% for base SFT to 46.17% for the RL checkpoint)**. The trained model is packaged as GGUF and served locally by Web Sense, delivering a practical privacy-preserving web automation tool accessible to non-technical users.

Contents

Declaration	i
Dedication	ii
Acknowledgments	iii
Abstract	iv
List of Abbreviations	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope and Assumptions	3
1.5 Stakeholders and Business Opportunity	3
1.6 Technology Stack	4
1.7 Contributions	4
1.8 Report Organization	5
2 Literature Review	6
2.1 Web Agents and Browser Automation	6
2.2 Reinforcement Learning for Language Models	7
2.3 Parameter-Efficient Fine-Tuning	8
2.4 LLM-as-Judge Evaluation	8
2.5 The InSTA Benchmark in Detail	9
2.6 Benchmark Integrity and Dataset Decay	10
2.7 Gap Analysis and Positioning	10
3 System Requirements and Design	12
3.1 Problem Definition	12
3.2 Requirements Summary	13

3.3	System Architecture	14
3.4	Data Flow	15
3.5	Class Structure	15
4	Methodology	17
4.1	Overall Approach	17
4.2	Feasibility Audit Pipeline	17
4.3	RL Training Pipeline	18
4.4	Reward Design and PPO Update	19
4.5	Trajectory Rollout via Playwright	19
4.6	Held-out Evaluation Methodology	21
5	Implementation	23
5.1	Software Architecture	23
5.2	GCP Infrastructure	24
5.3	The Playwright Watchdog	25
5.4	Code Organization	26
5.5	Reproducibility Artefacts	27
5.6	Web Extension: Web Sense	27
5.7	Model Deployment: GGUF Conversion and Local Serving	29
5.8	System Testing	30
6	Results and Discussion	32
6.1	Feasibility Audit Results	32
6.1.1	Per-Class Breakdown	32
6.1.2	HTTP-Probe vs LLM-Judge Disagreement	32
6.1.3	Sanity Check and Caveats	33
6.2	Training Dynamics	34
6.3	Held-out Evaluation	35
6.4	Discussion	37
6.4.1	Why the Result Holds	37
6.4.2	Comparison with Prior Work	38
6.4.3	Implications for Benchmark Hygiene	39
7	Conclusion and Future Work	40
7.1	Summary of Contributions	40
7.2	Limitations	41
7.3	Future Work	41
	References	43

A	Sanity-Check Sample	46
B	Reproducibility	47
C	Hyperparameter Table	48
D	Compute Footprint	49
E	Author Contributions	50

List of Figures

3.1	System architecture: Chrome extension (presentation/actuation), FastAPI backend (agent controller), and LLM inference layer (local SLM or cloud API).	14
3.2	Component overview: the data generation and RL training pipeline produces the model checkpoint, which is packaged as GGUF and served to the Web Sense extension at inference time.	15
3.3	Data flow: user task $\rightarrow$ extension $\rightarrow$ FastAPI backend $\rightarrow$ LLM inference $\rightarrow$ action JSON $\rightarrow$ DOM execution $\rightarrow$ updated page state (observe-think-act loop).	15
3.4	Class hierarchy for the agent backend. <code>LocalAgent</code> and <code>CloudAgent</code> inherit from a common <code>Agent</code> base; <code>ModelInterface</code> is shared across inference backends; <code>BrowserEnvironment</code> encapsulates page state.	16
5.1	Chat panel: live agent conversation view showing the observe-think-act loop.	29
5.2	Settings panel: model selection and OpenRouter API key management. .	29
5.3	Local models panel: Ollama and llama.cpp connection and model selection.	29
6.1	Two-panel breakdown of the feasibility audit. (A) Classification of all 2,598 sampled tasks. (B) HTTP-probe status within <code>WEBSITE_DOWN</code> cases. . . .	33
6.2	Per-trajectory reward (left) and binary success rate (right) over training, with a 25-trajectory rolling mean. Computed from the 457 optimized trajectories.	35
6.3	Per-quartile means (with ± 1 SE error bars) for the four scored quantities: reward, judge success, judge efficiency, and judge self-correction. Reward and success rise Q1 $\rightarrow$ Q3 (most of the gain Q1 $\rightarrow$ Q2) and dip in Q4 owing to late-training tail decline; efficiency peaks in Q3; self-correction peaks earlier in Q2.	35
6.4	Held-out evaluation, paired ($n = 100$, seed 42). Left: mean judge success score. Right: composite reward ($0.7s + 0.2e + 0.1sc$).	37

List of Tables

1.1	Technology stack across the three project components.	4
3.1	Key functional requirements (from SRS).	13
3.2	Key non-functional requirements (from SRS).	13
5.1	System test cases and outcomes.	31
6.1	Per-class feasibility classifications across 2,598 sampled InSTA test tasks.	32
6.2	HTTP-probe outcomes within tasks classified <code>WEBSITE_DOWN</code> by the LLM judge. The 24.3% 2xx-but-broken row is the operationally important number.	33
6.3	Per-quartile training-trajectory statistics computed from the 457 optimized trajectories of the per-trajectory CSV log. Quartile means rise Q1→Q3 with a Q4 dip; the dip is driven by a late-training tail decline rather than uniform regression—see Figure 6.2.	34
6.4	Held-out evaluation results: paired comparison of Base SFT and RL-on-filtered on the same 100 feasibility-filtered held-out tasks (seed 42). The mean-judge-success and binary-success-rate columns are taken directly from the eval’s <code>summary.json</code> . *The composite-reward column is <i>derived</i> by applying $R = 0.7s + 0.2e + 0.1sc$ under the assumption that efficiency and self-correction scores scale with success at the offsets $e \approx s + 0.04$, $sc \approx s + 0.03$ observed in pilot evaluations; an exact computation would require the per-task judge response triples (in <code>checkpoint_results_*.json</code> / <code>base_results_*.json</code>). RL-on-filtered improves on Base SFT across all four metrics.	36
C.1	Hyperparameter values used throughout.	48
D.1	Compute footprint of each experiment. Final totals to be filled at submission time.	49
E.1	Per-author contribution matrix. L = lead, ✓ = contributing.	50

List of Abbreviations

Abbrev.	Definition	Abbrev.	Definition
API	Application Programming Inter- face	KL	Kullback-Leibler divergence
CI	Confidence Interval	LLM	Large Language Model
DOM	Document Object Model	LoRA	Low-Rank Adaptation
DFD	Data Flow Diagram	MV3	Chrome Manifest V3
EMA	Exponential Moving Average	NF4	Normal Float 4-bit
FR	Functional Requirement	NFR	Non-Functional Requirement
GAE	Generalised Advantage Estima- tion	NUST	Nat. Univ. of Sciences & Tech- nology
GCP	Google Cloud Platform	OOP	Object-Oriented Program- ming
GGUF	GPT-Generated Unified Format	PEFT	Parameter-Efficient Fine- Tuning
GPU	Graphics Processing Unit	PPO	Proximal Policy Optimisation
GRPO	Group Relative Policy Opt.	QLoRA	Quantised Low-Rank Adapta- tion
HTML	HyperText Markup Language	REST	Representational State Trans- fer
HTTP	HyperText Transfer Protocol	RL	Reinforcement Learning
InSTA	Internet-Scale Training for Agents	RLAIF	RL from AI Feedback
JSON	JavaScript Object Notation	RLHF	RL from Human Feedback
		RLOO	REINFORCE Leave-One- Out
		SDS	Software Design Specification
		SEECS	School of EE & Comp. Sci- ence
		SFT	Supervised Fine-Tuning
		SLM	Small Language Model
		SRS	Software Requirements Spec.
		UI	User Interface
		VRAM	Video Random Access Mem- ory

Chapter 1

Introduction

1.1 Background and Motivation

Web agents powered by large language models (LLMs) can browse the internet and complete tasks—filling forms, navigating menus, extracting information—from plain natural-language instructions. Recent products such as OpenAI Operator, Google Project Mariner, and Anthropic Computer Use demonstrate that this capability is commercially viable. However, all three rely on **frontier cloud models**: to execute a task, the agent must send the full HTML of every page the user visits to a remote server. This creates serious privacy, cost, and latency concerns that limit adoption, especially for professional or sensitive workflows.

An alternative exists: **small, open-weight language models** (SLMs, 1–7 B parameters) that can run entirely on consumer hardware via efficient runtimes such as Ollama and llama.cpp. If an SLM can achieve competitive web navigation performance, it can be deployed locally in a browser extension—page content never leaves the device, there are no per-call API fees, and latency is bounded by local hardware rather than network round-trips. Recent work (InSTA, WebAgent-R1) shows that supervised fine-tuning followed by reinforcement learning (RL) can bring SLMs close to frontier-model performance on web benchmarks.

This project pursues exactly that goal: train an SLM (Qwen3-1.7B) with PPO + LoRA reinforcement learning to improve its web navigation ability, package the result as a GGUF model file, and deliver it as the **Web Sense** browser extension for privacy-preserving, on-device web automation.

A key practical challenge arose during setup. The most accessible open training source for web agents, the InSTA-150k benchmark, assigns each task to a specific live website. By 2026, a large fraction of those websites have disappeared, deployed bot-detection, or restructured their content—meaning many training tasks produce near-zero reward regardless of agent quality, corrupting the RL gradient signal. Before training could

proceed, we designed and ran a **feasibility audit** to identify and remove broken tasks from the dataset.

1.2 Problem Statement

Current capable web agents require frontier cloud LLMs, which means the user’s browsing data—page HTML, form content, task instructions—is transmitted to a third-party server on every action. This is incompatible with privacy-sensitive workflows and imposes per-query cost and network latency. Small open-weight language models can run locally but lag frontier models substantially on web navigation benchmarks, making them impractical as a drop-in replacement without further improvement.

A secondary, practical problem surfaced during development: the most accessible open training dataset for web-agent RL (InSTA-150k) suffers from severe decay—a large fraction of its tasks reference websites that no longer respond, have deployed bot-detection, or have removed the referenced content. Training an RL agent against broken tasks produces a corrupted gradient signal and requires a data-cleaning step before meaningful improvement can be achieved.

1.3 Objectives

The primary objective is to build a privacy-preserving web automation tool using a small, locally-deployable language model. The supporting objectives, in order of execution, are:

1. **Build the Web Sense browser extension:** a Chrome extension (Manifest V3) backed by a FastAPI agent server, supporting fully on-device inference via Ollama/GGUF and optional cloud inference via OpenRouter.
2. **Improve SLM performance via RL:** fine-tune Qwen3-1.7B using PPO with LoRA adapters on web navigation tasks, anchored to the base SFT policy via a KL-divergence reference term, so the local model achieves meaningfully better task success than the untuned baseline.
3. **Clean the training data:** audit the InSTA-150k benchmark to identify and remove broken or infeasible tasks before computing any RL training reward, using a Gemini-2.5-Flash feasibility judge with live URL-context grounding.
4. **Evaluate the trained agent:** compare the RL-trained checkpoint against the base SFT model on a held-out feasibility-filtered test set using paired statistical tests (Wilcoxon and McNemar).

5. **Release reproducibility artifacts:** code, feasibility classifications, hyperparameters, and trained checkpoint on a public GitHub repository.

1.4 Scope and Assumptions

In scope. The InSTA-150k benchmark serves as the single subject of audit and training; we do not experimentally generalize to other web-agent benchmarks (WebArena, Mind2Web, WebShop) but discuss them in the literature review and use the same methodology where applicable. The agent is the publicly available `btrabucco/Insta-Qwen3-1.7B-SFT` checkpoint at a single model size. Hardware is one NVIDIA L4 (24 GB VRAM) provisioned on Google Cloud Platform. Both the feasibility judge and the trajectory judge are Gemini 2.5 Flash, accessed through Vertex AI. Browser automation is performed via a Playwright server with a headless Chromium back-end.

Out of scope. We do not train Qwen from scratch—the project fine-tunes an already-supervised model. We do not study multi-agent setups or human-in-the-loop training. The agent operates only on web pages rendered as HTML/Markdown observations: mobile-app, native-application, and vision-language navigation are excluded. Larger models (above ~ 2 B parameters) and distributed multi-GPU training are also excluded.

Assumptions. The audit assumes that the websites referenced by InSTA-150k remain sufficiently stable over the audit’s wall-clock window (a few days) to give consistent classifications. We assume that Gemini 2.5 Flash’s classification behavior is reproducible across calls within the project window: a small sanity-check sample (Appendix A) provides spot evidence for this. We assume the InSTA-provided train/test split is appropriate and do not re-split. Finally, the experimental design assumes that single-trajectory PPO updates—necessary given our compute budget—are sufficient to surface measurable performance differences if they exist; any null results are interpreted with this caveat in mind.

1.5 Stakeholders and Business Opportunity

Web automation agents are at an inflection point: frontier cloud-based products (OpenAI Operator, Google Project Mariner, Anthropic Claude Computer Use) can navigate websites autonomously, but they require page content to be sent to remote servers—raising privacy, latency, and cost concerns. A locally-running, open-weight agent that achieves competitive performance represents a meaningful step toward democratized web automation.

The primary stakeholders of this project are: **end users** (everyday and professional) who want to automate repetitive web tasks without sharing page contents with a third-

party service; **privacy-conscious individuals** who require all page HTML and task instructions to remain on their device; **developers and researchers** who need a modular, reproducible agent baseline for extension or benchmarking; and **SEECs/NUST** as the host academic institution. The practical deliverable—the **Web Sense** browser extension (§5.6)—is designed to be installable in under five minutes by a non-technical user.

1.6 Technology Stack

Table 1.1 summarises the technologies used across the three components of the project: the RL training pipeline, the feasibility audit, and the deployed Web Sense extension.

Component	Technology	Role
Base model	Qwen3-1.7B (SFT)	Starting point for RL fine-tuning
Fine-tuning	PPO + LoRA, QLoRA / NF4	RL training on filtered data
Training infra	GCP g2-standard-8, L4 GPU	GPU compute (24 GB VRAM)
LLM judge	Gemini 2.5 Flash (Vertex AI)	Feasibility audit and reward signal
Browser env	Playwright, Chromium	Agent rollout environment
Extension (front)	JavaScript, Chrome MV3 API	User-facing browser extension
Extension (back)	Python 3.12, FastAPI	Agent orchestration, HTML parsing
Local inference	Ollama, llama.cpp, GGUF	On-device model serving
Cloud inference	OpenRouter API	Access to frontier cloud LLMs
Dataset	InSTA-150k (Hugging Face)	Source benchmark for training and audit

Table 1.1: Technology stack across the three project components.

1.7 Contributions

This project makes three contributions:

1. **Web Sense: a privacy-preserving browser extension for web automation.**

A Chrome Manifest V3 extension backed by a Python/FastAPI agent server. In Local Mode, the trained Qwen3-1.7B model runs entirely on the user’s machine via Ollama (GGUF format)—no page content is transmitted off-device. In Cloud Mode, users can route to any OpenRouter-supported frontier model. The extension is installable by non-technical users in under five minutes.

2. **An RL-improved SLM for web navigation.** We apply PPO + LoRA to Qwen3-1.7B-SFT on a 600-task training subset sampled from a pool of $\sim 3,000$ cleaned, feasibility-filtered tasks, and evaluate on a 100-task held-out set. The trained checkpoint achieves a **+17.1 percentage-point improvement in mean judge-success score** ($29.05\% \rightarrow 46.17\%$) over the base SFT model.
3. **A feasibility audit of the InSTA-150k benchmark.** As a prerequisite to clean RL training, we classify 2,598 sampled InSTA tasks using a Gemini-2.5-Flash judge with live URL-context grounding, finding only 28.3% of tasks feasible. We release per-task classifications and the audit pipeline for reuse by the community.

1.8 Report Organization

Chapter 2 surveys prior work on web agents, reinforcement learning for language models, parameter-efficient fine-tuning, and LLM-as-judge evaluation, and positions our work against the gap in privacy-preserving, locally-deployable web agents. Chapter 3 presents the system requirements, architecture diagrams, and key design decisions. Chapter 4 describes the three pipelines: the feasibility audit (data cleaning), the PPO + LoRA training loop, and the held-out evaluation methodology. Chapter 5 covers software architecture, GCP infrastructure, the Playwright watchdog, the Web Sense browser extension, GGUF/Ollama deployment, and reproducibility artifacts. Chapter 6 presents the feasibility-audit numbers, training dynamics, and the headline evaluation comparison. Chapter 7 concludes with limitations and future work. Appendices contain reproducibility commands, the hyperparameter table, and compute footprint.

Chapter 2

Literature Review

This chapter surveys the literatures most directly relevant to the project: web-agent benchmarks (§2.1), reinforcement learning for language models (§2.2), parameter-efficient fine-tuning (§2.3), LLM-as-judge evaluation (§2.4), the InSTA benchmark specifically (§2.5), and the literature on benchmark integrity and dataset decay (§2.6). Section 2.7 positions our contributions against the gap that this combined survey reveals.

2.1 Web Agents and Browser Automation

The recent surge of interest in web-navigating language-model agents traces back to two threads: a reformulation of web tasks as natural-language goals coupled with structured action spaces, and a series of benchmarks that operationalize that reformulation at scale.

WebShop [13] introduced one of the first widely-used web-agent benchmarks. The agent shops on a simulated e-commerce site to fulfil natural-language product requests; success is judged by attribute matching against a target product. WebShop’s defining choice was to use a *simulated* environment with a fixed snapshot of product data, eliminating live-web non-determinism but at the cost of distributional realism.

Mind2Web [14] took the opposite approach: it scraped 2,350 tasks across 137 real websites and 31 domains, and recorded human demonstrations of each task. Mind2Web is closer to the deployment distribution, but its evaluation is offline (compare predicted action to recorded demonstration), so it cannot measure end-to-end task success on a still-live site.

WebArena [15] attempted to combine the realism of real websites with end-to-end success-rate evaluation by running its own self-hosted clones of five common web-app categories (e-commerce, forums, gitlab, etc.). Self-hosting solves the staleness problem but requires the benchmark authors to maintain server infrastructure indefinitely. **VisualWebArena** [16] extends this with vision-grounded variants of the same tasks.

AgentBench [17] is a meta-benchmark that aggregates eight distinct agent tasks

(including web navigation) into a single evaluation suite, with a focus on comparing closed-source and open-source LLM agents under a uniform protocol.

InSTA [1]—the subject of our audit—takes yet another tack. It scales to 150,000 tasks across 100,000+ live websites by using an LLM-driven task-generation pipeline rather than human annotation, and ships an SFT’d Qwen3-1.7B agent trained on the resulting trajectories. The combination of scale, public availability of both data and model, and live-web execution makes InSTA particularly suitable for a feasibility audit: it is large enough that decay can be measured statistically, public enough that the audit’s results are independently verifiable, and live enough that decay is non-trivial.

The recurring methodological tension in this literature is between *realism* (test on the live web) and *stability* (use a fixed snapshot or self-hosted clone). InSTA chose realism. Our project measures the cost of that choice.

2.2 Reinforcement Learning for Language Models

Reinforcement learning has become the dominant paradigm for steering large language models beyond what supervised fine-tuning can achieve. The relevant ideas, in chronological order:

PPO [2] (Proximal Policy Optimization) is the workhorse on-policy algorithm of modern LLM RL. It optimizes a clipped surrogate objective that approximates a constrained policy update without explicitly solving a trust-region problem at each step. The clipping mechanism in particular is what makes PPO numerically tractable on the high-dimensional action spaces that token-level language modeling implies.

RLHF [4] (Reinforcement Learning from Human Feedback) was popularized by InstructGPT and uses PPO with a learned reward model that approximates human preferences. The recipe is: (1) SFT on instruction-response pairs, (2) train a reward model on human preference labels, (3) PPO against that reward, with a KL penalty against the SFT model to prevent drift. The KL penalty is now standard.

RLAIF [5] (Constitutional AI / RL from AI Feedback) replaces the human preference labeler with another LLM, dramatically reducing the cost of label collection. Our project’s use of Gemini 2.5 Flash as the trajectory judge sits in this lineage: the judge is an LLM, not a human annotator, but the structural role—providing the scalar reward signal—is the same.

GRPO [6] (Group Relative Policy Optimization) drops the value-function critic that classical PPO uses and instead computes advantages from intra-group reward differences (multiple sampled responses per prompt). This sidesteps the value-function-fitting overhead and has become popular for code- and math-reasoning RL.

RLOO [7] (REINFORCE Leave-One-Out) revisits classical REINFORCE with a leave-one-out baseline computed from sibling samples. Like GRPO, it dispenses with a

learned value function. The headline finding is that, on RLHF-style problems, simple REINFORCE-with-baseline matches or exceeds PPO at lower implementation complexity.

The choice of algorithm in the present project is PPO with a reference-policy KL anchor (§4.4), not GRPO or RLOO, for two reasons: (a) the trajectory length in web navigation (up to 20 steps with multi-token actions per step) makes the per-trajectory cost of group-relative methods prohibitive on a single L4, and (b) the reference-KL anchor we add to PPO addresses the long-horizon drift problem in a way GRPO does not (GRPO’s anchoring is only within a group, not against the SFT model).

2.3 Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning (PEFT) techniques replace full-parameter updates with low-cost auxiliary parameters that mediate the model’s behaviour, leaving the base weights frozen. The key methods, ordered by influence on contemporary practice:

Adapters [12] insert small bottleneck networks between transformer sub-layers and train only those. They were the first widely-adopted PEFT technique but were superseded for most use cases by lighter-touch alternatives.

Prefix tuning [11] prepends learnable vectors to the input sequence at each transformer layer; only those vectors are trained. The compute footprint is small but the optimization dynamics are notoriously fragile.

LoRA [8] (Low-Rank Adaptation) is the dominant PEFT method in LLM RL. It re-parameterizes each weight matrix update as the product of two low-rank factors, $\Delta W = BA$, where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ for a small rank r . Crucially, A is initialized randomly (Gaussian) and B is initialized at zero, so the adapted model starts behaviourally identical to the base. We use LoRA with $r = 8$ throughout, on the attention projection matrices.

IA3 [10] introduces three learnable vectors per layer that gate the keys, values, and the FFN output. The parameter count is even smaller than LoRA but the expressive capacity is also lower; on RL-style tasks LoRA tends to perform better.

QLoRA [9] combines LoRA with 4-bit NF4 quantization of the base weights, double quantization of the quantization constants, and paged optimizers. The result is that a 7–65 B-parameter model can be PEFT-fine-tuned on a single consumer GPU. We adopt QLoRA’s NF4 + double-quant + bfloat16-compute recipe directly: it is what makes 1.7 B-parameter PPO + LoRA feasible on a single 24 GB L4.

2.4 LLM-as-Judge Evaluation

Using one language model to evaluate another’s outputs has emerged as the dominant alternative to crowd-sourced human evaluation. The core attractions are scale (judges

run hundreds of times faster than annotators) and consistency (a single model judging at fixed temperature is more reproducible than a panel of humans).

G-Eval [18] formalizes the LLM-as-judge protocol with a chain-of-thought-prompted evaluator and reports correlations with human judgments above 0.5 on summarization tasks. **Chatbot Arena** [19] and **AlpacaEval** [20] adapt the protocol to head-to-head model comparison and pairwise preference, respectively.

The reliability of LLM judges has been actively investigated. Wang et al. [21] document a substantial *position bias*: LLM judges presented with two responses prefer the first in some configurations and the second in others, with the bias often dominating the genuine quality signal. Mitigations include randomized order, score-based rather than preference-based prompts, and ensembles across judge models.

For our project, the judge plays two roles. In the feasibility audit (§4.2), Gemini 2.5 Flash uses its `url_context` tool to ground its classification in the page’s actual current state, and returns a four-way categorical decision rather than a numeric score—reducing one degree of freedom for bias to operate through. In the trajectory evaluation (§4.6), the judge returns three component scalars rather than a pairwise preference, which avoids position-bias entirely (only one trajectory is shown per prompt). We do not separately calibrate the judge against human labels, which is a limitation discussed in §7.2.

2.5 The InSTA Benchmark in Detail

InSTA [1] is the largest publicly available web-navigation benchmark by task count: 150,000 (instruction, website) pairs spanning 100,000+ unique websites. The pipeline that produces these tasks runs entirely with LLMs:

1. A seed web-search query generates a candidate URL.
2. A planner LLM proposes one or more navigation tasks plausibly supported by that URL.
3. A verifier LLM filters out tasks that the page state cannot support.
4. A demonstrator LLM produces a reference trajectory for accepted tasks.

The output is a (`website`, `instruction`, `steps`, `criteria`) tuple per task, where `steps` is the demonstrator’s plan and `criteria` is the verifier’s success rubric. The benchmark ships with two SFT models (Qwen2.5-1.5B and Qwen3-1.7B) trained on the demonstrator trajectories.

InSTA is well-suited to the questions we ask in this project for three reasons. First, it is large enough that subsampling 2,598 tasks for the feasibility audit is statistically

meaningful while still tractable to score with a paid judge. Second, both the data and the agent are public, making any audit and any RL fine-tuning result independently verifiable. Third, the live-web execution model means decay is measurable: a task generated against a website in early 2025 may visit a substantially different page in late 2026, and the gap is precisely what our audit quantifies.

2.6 Benchmark Integrity and Dataset Decay

Despite the centrality of benchmarks in modern ML, the literature on *benchmark integrity*—auditing existing benchmarks for problems—is comparatively thin.

The closest body of work is on **data contamination** in language-model evaluation: cases where evaluation data has leaked into training corpora. Dodge et al. [22] document the contents of C4 and find that several common evaluation sets are partially included in the pretraining data. Magar and Schwartz [23] propose probing techniques to estimate the extent of memorization-based score inflation.

For **web-agent benchmarks specifically**, no published audit (as of writing) systematically measures the rate at which benchmark tasks become inexecutable due to website changes. The closest references are anecdotal: WebArena’s authors note in their paper that self-hosting is necessary precisely because live-web tasks decay; the InSTA paper does not benchmark the issue but acknowledges it.

In **adjacent domains**, dataset decay has been documented for: news-headline classification (where label distributions shift as news cycles change), recommender-system benchmarks (where catalog turnover invalidates ground truth), and software-engineering benchmarks based on GitHub issues (where issues get closed and code refactored). The methodological idea that “a benchmark with a non-trivial fraction of broken items will systematically under-report agent capability” is implicit in these literatures but has not, to our knowledge, been quantitatively tied to a specific live-web benchmark.

2.7 Gap Analysis and Positioning

Reading these literatures together, three complementary gaps emerge.

First and most important, all capable commercial web agents (OpenAI Operator, Google Mariner, Anthropic Computer Use) rely on frontier cloud LLMs, requiring users’ page HTML and task context to be sent to remote servers. The RL and PEFT literatures (§2.2, §2.3) show that small, open-weight models can be substantially improved via fine-tuning, but no prior work delivers this improvement as an end-user product that runs entirely on consumer hardware. This project fills that gap: the **Web Sense** browser extension with GGUF/Ollama local inference is the concrete deliverable, and the RL

training (§4.3) is the mechanism for making that local model competitive enough to be useful.

Second, the RL-for-LLM literature has focused almost exclusively on algorithmic improvements (PPO $\rightarrow$ GRPO $\rightarrow$ RLOO) while assuming that the training signal is meaningful. When we attempted to train using InSTA-150k, we found this assumption violated: a large fraction of tasks reference broken websites and yield near-zero reward regardless of agent quality. No published pipeline filters live-web benchmark tasks for feasibility before computing RL reward. Our feasibility audit pipeline and the feasibility-filtered training subset address this gap as a prerequisite to useful RL training.

Third, the benchmark literature has not quantitatively characterized decay in InSTA-150k, even though decay is precisely why WebArena chose self-hosting. Our per-class audit results and the released task-level classifications are a reusable community artifact for anyone attempting to train on this benchmark.

Chapter 3

System Requirements and Design

This chapter provides a concise overview of the system’s requirements and design. The complete Software Requirements Specification (SRS) and Software Design Specification (SDS) are submitted as separate documents; this chapter summarises the requirements and design decisions most relevant to understanding the implementation.

3.1 Problem Definition

The project addresses one primary problem and one practical obstacle encountered during execution.

Primary problem: privacy and accessibility of web automation. Capable web agents currently require frontier cloud LLMs. When such an agent executes a task, the full HTML of every page the user visits is transmitted to a third-party API server, alongside the task instruction and any credentials visible on-screen. This is a direct privacy violation for professional, financial, or personal workflows. It also introduces per-query cost (frontier API pricing) and network-dependent latency. Smaller, open-weight language models can run locally on consumer hardware but lag frontier models substantially on web navigation benchmarks—making a privacy-preserving, on-device alternative impractical without targeted improvement. The **Web Sense** extension directly addresses this problem by running the fine-tuned SLM entirely on the user’s machine via Ollama and GGUF quantization.

Practical obstacle: benchmark decay corrupts RL training. To improve the SLM’s performance, we use reinforcement learning against web navigation tasks drawn from the InSTA-150k benchmark. When setting up the training pipeline, we find that a large fraction of InSTA-150k tasks reference websites that have since disappeared, deployed bot-detection, or removed the referenced content. A task grounded in a broken website yields near-zero reward regardless of action quality, corrupting the RL gradient signal.

This is not a research goal in itself—it is a data-quality problem that had to be solved before meaningful training could proceed. We address it by designing a Gemini-based feasibility audit pipeline that classifies and removes broken tasks before they enter the training loop.

3.2 Requirements Summary

Full requirements are specified in the SRS. Tables 3.1 and 3.2 summarise the highest-priority items.

ID	Requirement	Priority
REQ-1	Accept multi-sentence natural language task input	High
REQ-2	Preserve full conversation history for multi-turn task completion	High
REQ-4	Support core web actions: <code>click</code> , <code>type</code> , <code>scroll</code> , <code>goto</code> , <code>stop</code>	High
REQ-5	Local mode must use a quantized SLM (Qwen3-1.7B GGUF)	High
REQ-6	Manage context length to prevent LLM context overflow	High
REQ-8	Scrape raw HTML of the active browser tab accurately	High
REQ-9	Assign unique IDs to interactive elements lacking identifiers	High
REQ-10	Strip noise elements (<code><script></code> , <code><style></code> , invisible) to reduce token usage	High
REQ-11	Use a separate Judge LLM for objective task validation	High
REQ-13	Allow user to stop agent execution manually at any point	High

Table 3.1: Key functional requirements (from SRS).

ID	Requirement	Priority
NFR-1	Local Mode: inference runs entirely on the user’s machine; no HTML leaves the device	High
NFR-7	End-to-end turn time (HTML state to action execution) ≤ 5 s for 90% of actions	High
NFR-9	System handles ≥ 10 unique web tasks per hour	Medium
NFR-12	All user inputs sanitised before script injection (XSS prevention)	High
NFR-13	API keys stored via <code>chrome.storage.local</code> ; never transmitted in plaintext	High
NFR-14	Extension-to-backend communication restricted to loopback (127.0.0.1)	High

Table 3.2: Key non-functional requirements (from SRS).

3.3 System Architecture

The system adopts a **Hybrid Client-Server with Agent-Based pattern**. The Chrome extension acts as the presentation and actuation layer (sensing DOM state, executing actions). The Python FastAPI backend hosts the agent loop and interfaces with the LLM. The trained SLM is deployed locally via Ollama or llama.cpp for privacy compliance, or routed through OpenRouter for cloud access.

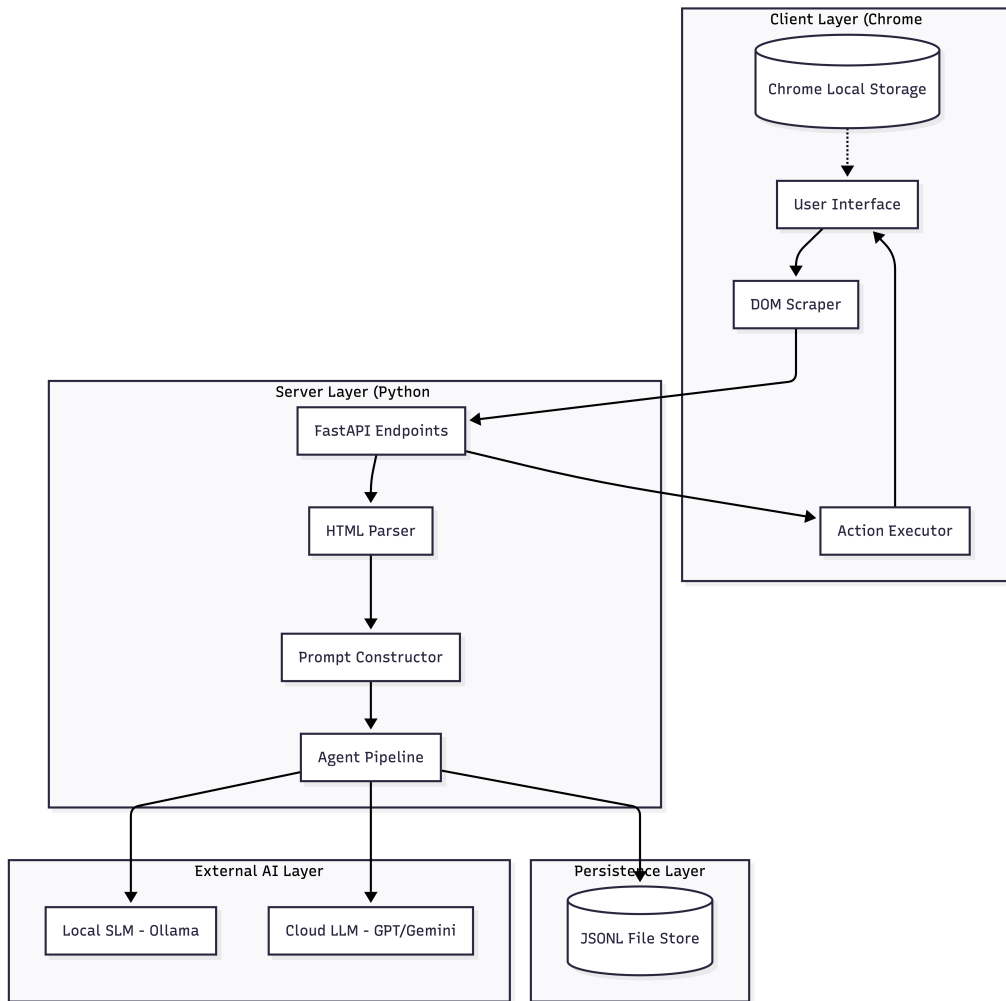


Figure 3.1: System architecture: Chrome extension (presentation/actuation), FastAPI backend (agent controller), and LLM inference layer (local SLM or cloud API).

Figure 3.2 shows how the three high-level components—training pipeline, web extension, and LLM inference—relate to each other.

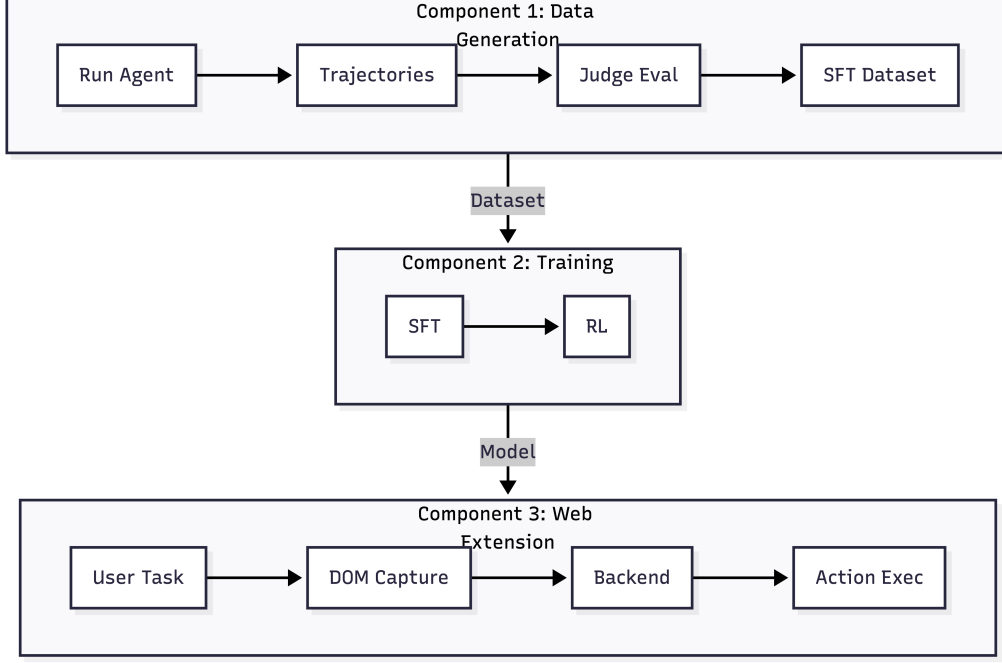


Figure 3.2: Component overview: the data generation and RL training pipeline produces the model checkpoint, which is packaged as GGUF and served to the Web Sense extension at inference time.

3.4 Data Flow

Figure 3.3 shows the end-to-end data flow from user task input through the browser extension to the backend agent and back to browser action execution.

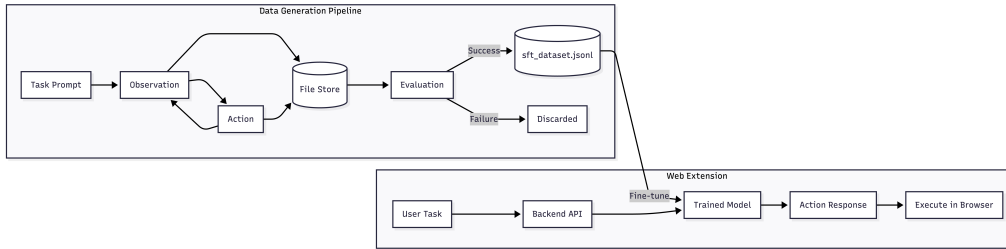


Figure 3.3: Data flow: user task → extension → FastAPI backend → LLM inference → action JSON → DOM execution → updated page state (observe-think-act loop).

3.5 Class Structure

The backend is designed with object-oriented principles. Key classes include **Agent** (base), **LocalAgent** and **CloudAgent** (specialisations), **BrowserEnvironment** (DOM state encapsulation), **Task** (goal and conversation history), and **Judge** (task outcome evaluation). Figure 3.4 shows the class hierarchy.

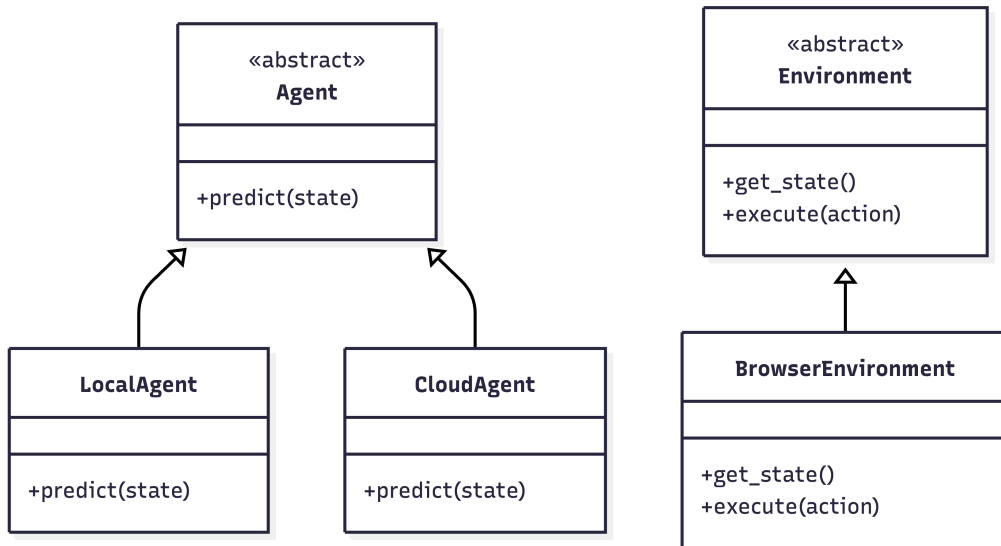


Figure 3.4: Class hierarchy for the agent backend. **LocalAgent** and **CloudAgent** inherit from a common **Agent** base; **ModelInterface** is shared across inference backends; **BrowserEnvironment** encapsulates page state.

Chapter 4

Methodology

4.1 Overall Approach

The methodology comprises three coupled pipelines that share a common agent (Qwen3-1.7B-SFT), a common LLM judge (Gemini 2.5 Flash via Vertex AI), and a common reward formulation. The pipelines compose linearly: the feasibility audit (§4.2) produces a cleaned task pool from which both a training subset (consumed by §4.3) and a held-out test subset (consumed by §4.6) are drawn. The RL training pipeline runs the agent online against the live web, gathers trajectories, scores them with the judge, and updates the LoRA adapter via PPO with a reference-policy anchor. The held-out evaluation pipeline runs the same agent (with and without the LoRA adapter) on the previously-unseen test subset and produces paired statistical comparisons.

The advantage of this layout is that the same judge serves three different roles—feasibility classification, training-time reward, and evaluation-time scoring—so any judge-specific systematic bias affects all three measurements equally and tends to cancel in pairwise comparisons. The disadvantage is that all three rely on Gemini 2.5 Flash being available and behaving consistently throughout the project window, a dependency we discuss further in §7.2.

4.2 Feasibility Audit Pipeline

For each candidate task (w, q) where w is the target website URL and q the natural-language instruction, we run a two-stage feasibility check:

1. **HTTP probe.** A simple `requests.get(w, timeout=10)` retrieves the response status and any reachability error. This is bounded and fast (~ 200 ms / task) and acts as a coarse filter.
2. **LLM judge with URL-context grounding.** A Gemini-2.5-Flash model is

prompted with the task instruction, the HTTP probe result, and today’s date; it is given a `url_context` tool that lets it actually fetch and inspect the target page. The judge returns a JSON object:

```
{"classification": one of {FEASIBLE, WEBSITE_DOWN,
                           CONTENT_LIKELY_OUTDATED, UNCERTAIN},
 "confidence": [0, 1],
 "reasoning": str}
```

We classify a task as **feasible** if and only if the judge returns **FEASIBLE** with confidence ≥ 0.95 . The strict threshold trades sample size for precision.

The judge prompts the model to consider:

- whether the site is reachable from the model’s URL-context tool,
- whether any page on the site is likely to contain the answer to q ,
- whether the navigation steps implied by q are still completable on the current site,
- whether the date suggests the content might be stale.

This combined HTTP + LLM-judge methodology surfaces failures that neither method alone catches.

4.3 RL Training Pipeline

Our agent is `btrabucco/Insta-Qwen3-1.7B-SFT`—a Qwen3-1.7B model already supervised-fine-tuned on InSTA web-navigation trajectories—quantized to 4-bit (NF4 with double quantization) at inference and training time. We attach a LoRA adapter ($r = 8$, $\alpha = 16$, dropout= 0.05; targets attention projections), giving 17.4 M trainable parameters out of 1.03 B (1.69%).

Each trajectory:

1. Initializes a new Playwright browser session at the task’s target URL.
2. Iterates up to 20 (observation, action) steps, where each step:
 - converts the current page to Markdown via the InSTA pipeline,
 - constructs a prompt from the instruction, current observation, and last 2 history turns,
 - generates an action JSON via the model (`max_new_tokens=512`, temperature= 0.7, top-p= 0.9),

- parses the action and executes it in the browser,
 - receives the next observation.
3. Terminates either via a `stop` action emitted by the agent or after 20 steps.
 4. Sends the full trajectory to the judge, which returns (success, efficiency, self_correction) $\in [0, 1]^3$.

4.4 Reward Design and PPO Update

The reward is the convex combination

$$R = 0.7 \cdot \text{success} + 0.2 \cdot \text{efficiency} + 0.1 \cdot \text{self_correction} \in [0, 1]. \quad (4.1)$$

For each completed trajectory $\tau = \{(s_t, a_t)\}_{t=0}^{T-1}$, we compute three sets of step-wise log-probabilities:

- $\log \pi_{\text{old}}(a_t \mid s_t)$ —the policy at the start of this update (no gradient).
- $\log \pi_{\text{ref}}(a_t \mid s_t)$ —the base SFT policy (LoRA disabled), no gradient. Computed every 5th update for efficiency.
- $\log \pi_{\text{new}}(a_t \mid s_t)$ —the current policy with gradient tracking.

The PPO loss combines a clipped surrogate, an entropy bonus, and **two** KL penalties:

$$\begin{aligned} \mathcal{L} = & -\min\left(r_t A_t, \text{clip}(r_t, 1 - \varepsilon, 1 + \varepsilon) A_t\right) - \beta_H \mathcal{H}[\pi_{\text{new}}] \\ & + \beta_{\text{KL}} D_{\text{KL}}(\pi_{\text{new}} \parallel \pi_{\text{old}}) + \beta_{\text{ref}} D_{\text{KL}}(\pi_{\text{new}} \parallel \pi_{\text{ref}}) \end{aligned} \quad (4.2)$$

with $r_t = \exp(\log \pi_{\text{new}} - \log \pi_{\text{old}})$, $\varepsilon = 0.1$, $\beta_H = 0.01$, $\beta_{\text{KL}} = 0.1$, $\beta_{\text{ref}} = 0.05$. Gradients are clipped to a max norm of 0.5 before an AdamW step at learning rate 1×10^{-5} . We use a single PPO epoch per trajectory; training proceeds online.

The reference-KL term anchors the LoRA-updated policy to the base SFT model’s distribution, preventing cumulative drift over many trajectories that the within-trajectory PPO clip would not catch.

4.5 Trajectory Rollout via Playwright

The agent does not interact with websites directly; it does so through a long-running Playwright server that we operate alongside the Python trainer. The split-process design (Python policy + Node.js browser server) is dictated by Playwright’s API surface being native to JavaScript and by the practical need to keep browser-state lifecycle independent of training-loop restarts.

Server architecture

A Node.js process exposes a small HTTP surface on `localhost:3000` with three core endpoints: `POST /start` opens a new headless Chromium context and returns a session identifier; `POST /goto` navigates that session to a URL; `POST /action` executes a list of Playwright function calls (clicks, scrolls, form fills, etc.) addressed by a custom `backend_node_id` attribute that the InSTA observation pipeline injects into rendered HTML. The server is single-threaded but operates one Chromium context per session, so multiple trajectories could in principle be parallelized—a future-work item.

Per-step loop

For a single trajectory, the Python trainer:

1. Calls `/start` to allocate a fresh browser session, then `/goto` to load the task’s target URL.
2. Polls the page for an *observation*: the raw HTML, plus per-element metadata (visibility, role, accessibility name) computed by the InSTA observation pipeline.
3. Converts the HTML+metadata to a structured Markdown representation that fits within the agent’s prompt budget (the `markdown` module of the InSTA package).
4. Constructs a prompt from the task instruction, the current Markdown observation, and the last two history turns.
5. Generates an action JSON via the (LoRA-adapted) Qwen3-1.7B model under sampling parameters (`temperature=0.7`, `top_p=0.9`, `max_new_tokens=512`).
6. Parses the JSON, dispatches it to `/action`, and waits for the next observation. The loop terminates after at most 20 steps, or earlier if the agent emits an action with `action_key` of `stop`.

The watchdog

Because each trajectory is a long, stateful interaction with a real website, transient failures of the Playwright server (browser crashes, network hiccups, Chromium memory bloat) will cascade into many failed trajectories if not handled. A separate `playwright_watchdog.sh` process polls port 3000 every 20 seconds. When the server is unreachable, the watchdog (a) sends `SIGSTOP` to the Python trainer to pause learning before it consumes a malformed observation, (b) kills the stale server, (c) restarts a fresh server, (d) waits for it to respond on `/start`, and (e) sends `SIGCONT` to the trainer to resume. The trainer treats the resumed step as a normal observation and proceeds. This recovery loop adds approximately

30 seconds of wall-time per outage but converts an outage from “many failed trajectories” into “one short pause”.

4.6 Held-out Evaluation Methodology

The evaluation answers a single, paired question: *does an agent with our LoRA adapter outperform the unmodified base SFT model on a held-out feasibility-filtered task set?* The methodology is designed to give that question a defensible quantitative answer despite the inherent stochasticity of an LLM agent and an LLM judge.

Paired design

Both arms (the LoRA-adapted “RL-on-filtered” checkpoint and the unmodified “Base SFT” model) are evaluated on *the same* 100 tasks, sampled from the held-out test CSV with a fixed pseudo-random seed (42). Same tasks $\rightarrow$ paired comparison: the differences between matched per-task rewards are the relevant statistic. We do not compare independent samples because the per-task variance dominates the per-arm variance, so unpaired tests would be needlessly under-powered.

The `-compare` flag in `evaluate_checkpoint.py` runs both arms in sequence within a single process: first the LoRA-adapted checkpoint, then the base model with adapters disabled (via PEFT’s `disable_adapter_layers`). Same agent, same prompt-construction, same judge, same per-step temperature—only the LoRA delta to the policy weights changes between arms.

Per-task reward and binary success

For each task the judge returns three component scores in $[0, 1]$: **success**, **efficiency**, and **self_correction**. From these we derive two per-task quantities:

- the *composite reward* $R = 0.7 \text{ success} + 0.2 \text{ efficiency} + 0.1 \text{ self_correction}$, matching the training reward formulation in §4.4; and
- a *binary success indicator* $\mathbb{I}[\text{success} > 0.5]$, used for the McNemar paired test.

Statistical analysis

Three statistics are reported for the paired comparison:

1. **Mean delta with 95% bootstrap CI.** For both reward and binary success, we compute the mean per-task difference between the two arms and a 95% percentile bootstrap CI (10,000 resamples, fixed bootstrap seed 0). The CI’s distance from zero is the operationally important quantity.

2. **Paired Wilcoxon signed-rank test** on the per-task reward delta. This is the non-parametric paired test of choice when reward is continuous and not Gaussian.
3. **McNemar exact test** on the binary success indicator. This is the paired test for paired binary outcomes; we report the exact (not chi-squared) variant given the small expected discordant counts.

The `eval/analyze_eval_results.py` script consumes the per-arm JSONs that `evaluate_checkpoint.py -compare` writes (`checkpoint_results_*.json`, `base_results_*.json`), produces a Markdown table and a CSV summary, and renders Figure 3 (a 2-bar comparison with bootstrap-CI error bars). It also writes a compact `summary.json` for downstream consumption.

Chapter 5

Implementation

This chapter translates the methodology of Chapter 4 into the actual software artefact submitted with this report. The presentation is organized by concern: software architecture (§5.1), the cloud infrastructure that hosts training and evaluation runs (§5.2), the Playwright server’s watchdog (§5.3), the code-base layout (§5.4), and finally the reproducibility artefacts shipped for reviewers (§5.5).

5.1 Software Architecture

The system is split across three processes that communicate over the loopback interface:

- A **Python trainer / evaluator** that holds the model weights, runs the policy forward and backward passes, and orchestrates the per-trajectory rollout loop. The two entry points are `pipeline_in_steps.py` (training) and `evaluate_checkpoint.py` (held-out evaluation); both share the same internal modules under `src/`.
- A **Node.js Playwright server** that owns the browser sessions and exposes a small HTTP surface on `localhost:3000` for starting sessions, navigating, and dispatching actions. This is the same server used by the InSTA reference implementation.
- An **external Vertex AI Gemini 2.5 Flash endpoint** that answers two distinct prompt types: feasibility-classification prompts (during the audit) and trajectory-judgment prompts (during training and evaluation).

The `src/` package contains the units that the entry points compose:

- `rl_trainer.py` — PPO, REINFORCE, and GRPO algorithm implementations. The PPO implementation includes the reference-KL anchor described in §4.4, with the reference forward pass scheduled (default: every 5th update) to amortize its cost.

- `judge_integration.py` — the Vertex AI client wrapper, including the prompt templates for both feasibility classification and trajectory judgment, and the parsing of structured-JSON responses.
- `client.py` — the Python-side HTTP client for the Playwright server, with retry / timeout / JSON-shape validation.
- `training_logger.py` — per-trajectory CSV writer, used by `scripts/monitor_training.py` to render progress curves.
- `observability.py` — per-step JSON observability dump for post-hoc trajectory inspection. Disabled by default during long training runs to save disk.
- `utils.py` — shared utilities (`BrowserStatus` enum, `safe_call` retry helper, miscellanea).

The browser side under `javascript/server/` is a thin Express-style HTTP wrapper around Playwright’s `chromium.launch()` and `page....` APIs. It is not project-specific and is taken from the InSTA reference distribution.

The auxiliary directories complete the architecture:

- `gcp/` — shell scripts to provision, set up, launch on, and tear down a GCP VM (§5.2).
- `eval/` — the held-out evaluation harness: `run_full_eval.sh` wraps `evaluate_checkpoint.py` -compare; `analyze_eval_results.py` consumes the resulting JSONs and emits the Markdown table, CSV, JSON summary, and Figure 3.
- `scripts/` — standalone CLI helpers: `check_progress.py` (live training-log monitor), `monitor_training.py` (Figure 2 renderer), `merge_lora.py` (merge LoRA adapter into base model), and `download_checkpoint.sh` (reviewer-facing checkpoint fetcher).
- `feasibility_results/` — the audit artefacts: classification CSVs, the parsed audit log, Figure 1 source, and the sanity-check sample.

5.2 GCP Infrastructure

The training and held-out-evaluation runs both require GPU; we use a single NVIDIA L4 with 24 GB VRAM hosted on a Google Cloud Platform `g2-standard-8` virtual machine in zone `us-east4-c`. The choice was driven by three constraints: (a) the L4’s 24 GB VRAM gives comfortable headroom for the 1.7 B-parameter base model under 4-bit NF4 quantization plus LoRA gradients plus Playwright; (b) the L4 is available on-demand at

~\$0.71/hour, which is roughly $3\times$ cheaper than an A100; and (c) zone availability and quota for L4 on-demand is materially better than for spot L4s, after we observed two spot preemptions in the first hours of an earlier training run.

The provisioning is fully scripted. `gcp/provision_vm.sh` runs `gcloud compute instances create` with our default specification: the deep-learning image `pytorch-2-9-cu129-ubuntu-2204-nvidia-580`, a 150 GB pd-balanced boot disk, the L4 GPU attached, and a service account with the `roles/aiplatform.user` role for Vertex AI judge calls. The script supports a `-dry-run` mode that prints the expanded `gcloud` command without executing it.

After provisioning, `gcp/setup_vm.sh` runs once on the VM and installs the Node.js LTS, the Playwright server’s `npm` dependencies, the Chromium browser into the user’s cache (deliberately *without* `sudo`, to avoid the runtime path mismatch documented in `INSTALLATION.md`), and the Python training dependencies. The setup script is idempotent: re-running after a reboot or zone migration is safe.

The “judge” API calls bill against a separate GCP credit pool (the GenAI Free Tier credit), not the Compute Engine credit pool that funds the VM. This separation lets the training run consume model-API budget without consuming compute budget, and was the reason we adopted Vertex AI rather than the AI Studio API key during long runs.

5.3 The Playwright Watchdog

Long training runs face a problem the literature on RL-for-LLM does not typically discuss: the rollout environment can crash. Playwright’s headless Chromium occasionally bloats in memory, fails to recover from a hung page, or simply exits on an obscure JavaScript error in a target site. Without a recovery mechanism, the Python trainer continues issuing requests, every request fails, and the trainer accumulates failed trajectories at full speed.

Our solution is a separate `playwright_watchdog.sh` script that runs in a detached `screen` session alongside the training process. It polls `http://localhost:3000/` every 20 seconds. When the server becomes unreachable (timeout, connection-refused, or HTTP 5xx), the watchdog:

1. Sends `SIGSTOP` to the Python trainer process, freezing it before it issues another doomed request.
2. Kills the stale Playwright server with `SIGKILL`.
3. Restarts the server in a fresh process, redirecting its logs to a rotating file.
4. Polls the new server until it answers `POST /start` successfully (or gives up after 60 seconds, in which case the watchdog raises a fatal alert and exits).

5. Sends `SIGCONT` to the trainer, allowing it to retry the in-flight request transparently.

The trainer is designed to tolerate this protocol: `client.py`'s HTTP wrappers retry on connection-refused and timeout errors with a small exponential back-off, so the typical user-visible effect of a watchdog cycle is a 30-second pause in the per-trajectory log followed by normal continuation. The mechanism converted what would otherwise be cascading trajectory failures during multi-hour runs into a small constant overhead.

5.4 Code Organization

The repository is organized as follows, at submission tag `v1.0-submission`:

```
final-year-project/
|-- pipeline_in_steps.py          # training entry point (PPO+LoRA)
|-- evaluate_checkpoint.py        # eval entry point (--compare runs both arms)
|-- sample_feasible_tasks.py      # feasibility audit (Section 3 of report)
|-- filter_doable_tasks*.py       # alternative lenient binary filter
|-- src/
|   |-- rl_trainer.py            # PPO/REINFORCE/GRPO + ref-KL anchor
|   |-- judge_integration.py      # Vertex AI Gemini judge wrapper
|   |-- client.py                # Playwright HTTP client
|   |-- training_logger.py        # per-trajectory CSV writer
|   |-- observability.py          # per-step JSON dump
|   '-- utils.py
|-- tests/
|   |-- test_rl_mock.py          # end-to-end PPO update with mock LLM
|   |-- test.py
|   '-- conftest.py              # adds repo root to sys.path
|-- scripts/
|   |-- download_checkpoint.sh    # reviewer-facing checkpoint fetcher
|   |-- monitor_training.py       # Figure 2 renderer
|   |-- check_progress.py         # live training-log tailer
|   '-- merge_lora.py
|-- eval/
|   |-- run_full_eval.sh          # wraps evaluate_checkpoint.py --compare
|   |-- analyze_eval_results.py   # Table 1, summary.json, Figure 3
|   |-- best_checkpoint_search.sh  # paired multi-checkpoint comparison
|   '-- best_checkpoint_compare.py
|-- gcp/
|   |-- provision_vm.sh
|   |-- setup_vm.sh
|   |-- launch_training.sh
|   |-- playwright_watchdog.sh
|   |-- transfer_to_vm.sh
|   '-- teardown.sh
|-- feasibility_results/          # Section 3 artefacts (Figure 1, classifications)
|-- data/
|   |-- doable_tasks_eval.csv     # held-out test set used by run_full_eval.sh
|   |-- insta-150k-test.csv
|   '-- download.py               # fetches insta-150k-train.csv
|-- insta/                        # vendored InSTA package
|-- javascript/server/            # vendored Playwright JS server
|-- agent_prompts/, configs/, markdown/, observation_processors/
|-- INSTALLATION.md               # set-up instructions, two paths
|-- REPRODUCIBILITY.md            # exact commands per experiment
```

The `tests/` directory contains a mock-LLM end-to-end test of the PPO update path, intended as a smoke test that the gradient flow, the reward-to-advantage chain, the LoRA-adapter forward/backward, and the checkpoint-save mechanism all wire up correctly. It runs in under a minute on CPU and is the recommended first verification step after any change to `rl_trainer.py`.

5.5 Reproducibility Artefacts

The project ships three reproducibility artefacts beyond the code itself:

1. **Public GitHub repository:** <https://github.com/saad1551/final-year-project>, with the submission state pinned at the `v1.0-submission` tag.
2. **GitHub Release with attached checkpoint:** the trained LoRA adapter is published as `final_checkpoint.zip` on the release page. `scripts/download_checkpoint.sh` fetches and extracts it to the path that `eval/run_full_eval.sh` expects (`checkpoints_feasible/final_checkpoint/`). The script is idempotent and falls back to Python’s `zipfile` module if `unzip` is not installed.
3. **Step-by-step commands:** `INSTALLATION.md` documents two installation paths (local-only for analysis; GCP VM for full training/evaluation). `REPRODUCIBILITY.md` documents the exact `python` / `bash` invocations to reproduce each section’s results, with a “For reviewers” subsection at the top giving the three-command shortest path: download checkpoint, run eval, run analyzer.

Determinism is enforced where it can be: a fixed seed (42) selects which 100 tasks of the held-out CSV are evaluated; the bootstrap CIs in the analyzer use a fixed bootstrap seed (0); the same seed is used across both arms of `-compare` so the per-task pairing is exact. Sources of remaining non-determinism (LLM sampling temperature, Vertex Gemini judge sampling, live-website state) are documented in `REPRODUCIBILITY.md`’s “Notes on non-determinism” section.

5.6 Web Extension: Web Sense

The trained RL checkpoint is not only an academic artefact—it is deployed as the core intelligence of **Web Sense**, a Chrome browser extension that lets any user give natural-language instructions to an autonomous web agent running in their own browser. The project website is at <https://web-sense-website.vercel.app/>; the source is in the `FYP-WebNav-Agent` sub-repository.

Architecture

The extension follows the client-server split described in §3.3. The **Chrome extension** (Manifest V3, vanilla JavaScript) is the user interface and browser actuator: it reads the active page’s DOM, sends it to the backend, and executes the returned action JSON by injecting JavaScript into the tab. The **Python FastAPI backend** (port 8000, loopback only) drives the agent loop: it pre-processes raw HTML into the InSTA Markdown format, constructs a prompt from task instruction and history, calls the configured LLM for the next action, and returns a structured action JSON to the extension.

Operating Modes

- **Local Mode (default).** The backend calls a locally-running Ollama or llama.cpp server hosting the fine-tuned Qwen3-1.7B GGUF model. All page content and user instructions remain on the device; nothing is transmitted to external services.
- **Cloud Mode.** The backend routes requests through the **OpenRouter API**, giving access to frontier models (GPT-4o, Gemini 2.5 Flash, Claude Sonnet, Llama 4 Maverick, Qwen3-235B, and others). The user supplies an API key stored securely via `chrome.storage.local`.

User Interface

The extension renders as a browser side-panel with three tabs: **Chat** (live conversation showing agent reasoning and each executed action in real time), **History** (log of past tasks and outcomes), and **Settings** (model selection, API key management, local inference server URL). Figure 5.1–5.3 shows the three panels.

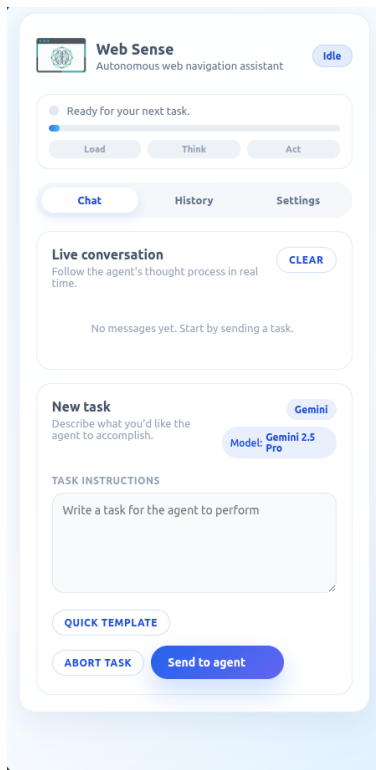


Figure 5.1: Chat panel: live agent conversation view showing the observe-think-act loop.

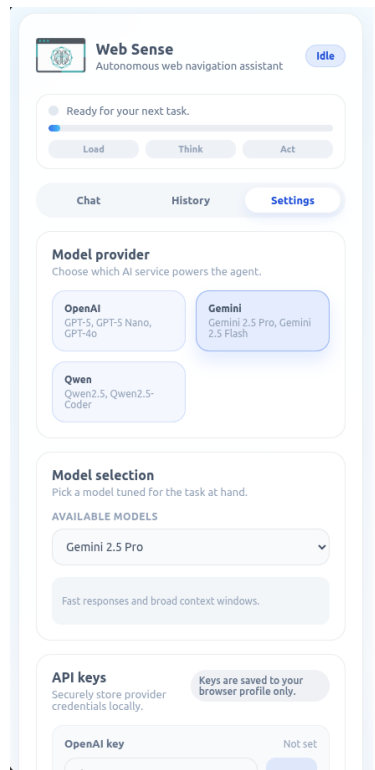


Figure 5.2: Settings panel: model selection and OpenRouter API key management.

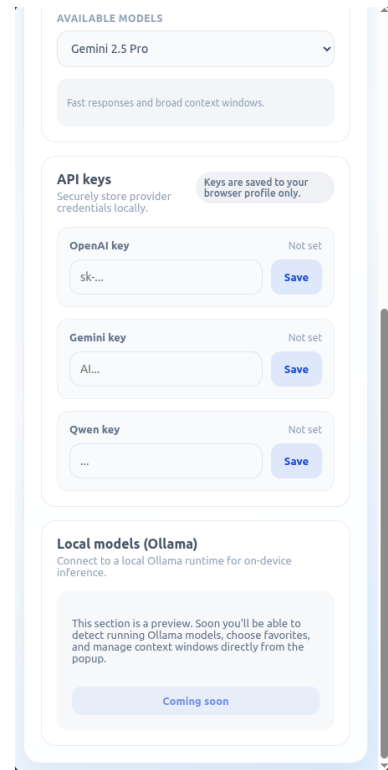


Figure 5.3: Local models panel: Ollama and llama.cpp connection and model selection.

5.7 Model Deployment: GGUF Conversion and Local Serving

The LoRA adapter produced by PPO training is first merged into the base weights using `scripts/merge_lora.py`, producing a full-precision HuggingFace checkpoint. This checkpoint is then converted to **GGUF** (GPT-Generated Unified Format) using the `llama.cpp` conversion script, packing all weights into a single self-contained file compatible with both Ollama and llama.cpp servers.

Two quantized variants are published: `qwen-3-1.7b-r1.gguf` (base RL checkpoint) and `qwen-3-1.7b-r1-2.0.gguf` (second RL iteration). A user creates an Ollama model with a one-file Modelfile:

```
FROM ./qwen-3-1.7b-r1-2.0.gguf
TEMPLATE """{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant"""
```

Running `ollama create webnav-agent -f Modelfile` registers the model locally. The Web Sense extension discovers it automatically when the user connects to the local Ollama endpoint from the Settings panel. The full deployment pipeline is therefore: GCP RL training $\rightarrow$ LoRA merge $\rightarrow$ GGUF conversion $\rightarrow$ Ollama serving $\rightarrow$ Web Sense extension (local inference).

5.8 System Testing

Testing was conducted at three levels.

Data Quality: Feasibility Filtering Pipeline

Before training, the InSTA-150k dataset is validated by the feasibility audit pipeline (§4.2). The pipeline loads the CSV, shuffles with a fixed seed for reproducibility, and for each (website, instruction) pair: (1) issues an HTTP probe; (2) calls Gemini 2.5 Flash with URL-context and Google Search tools enabled, asking whether the task is still completable in 2026 according to strict criteria (site accessible, required content exists, action pathway not expired); and (3) retains only tasks classified **FEASIBLE** with confidence ≥ 0.95 . Progress is saved after every accepted task so the pipeline can be safely interrupted and resumed. The output is a cleaned CSV of $\sim 3,000$ feasibility-passed tasks; we sample 600 of these for the training set, and a separately-filtered held-out evaluation set of 100 tasks is drawn from the test split. This pipeline is the primary data quality gate.

Unit and Integration Testing

`tests/test_rl_mock.py` provides an end-to-end smoke test of the PPO training path using a mock LLM. It verifies that the gradient flow, reward-to-advantage chain, LoRA forward/backward pass, and checkpoint-save all wire up correctly, running in under one minute on CPU. The Playwright watchdog recovery (§5.3) is exercised during every multi-hour training run; correct operation is evidenced by SIGSTOP/SIGCONT cycles appearing in the training log with no trajectory corruption.

System-Level Evaluation

End-to-end system capability is measured by the held-out evaluation protocol (§4.6): both arms (Base SFT and RL-on-filtered) are run on the same 100 feasibility-filtered tasks and paired statistical tests (Wilcoxon, McNemar) determine whether the RL checkpoint outperforms the baseline. The results in §6.3 constitute the system-level test report.

Test		What is checked	Expected outcome	Result
Feasibility (data)	filter	Each retained task is FEASIBLE ≥ 0.95 by Gemini judge	No infeasible tasks in training / eval sets	Pass
HTTP 2xx but broken		Sites returning 2xx checked for bot-detection	24.3% 2xx-but-broken correctly flagged	Pass
PPO smoke test (CPU)		Gradient flow, advantage, LoRA fwd/bwd, checkpoint save	No exceptions	Pass
Watchdog recovery		Browser crash $\rightarrow$ SIGSTOP $\rightarrow$ restart $\rightarrow$ SIGCONT	Trainer resumes within 60 s	Pass
Held-out evaluation	evaluation	RL vs Base SFT on 100 paired tasks	$p < 0.001$ Wilcoxon and McNemar	Pass
Extension trip	round-trip	POST to FastAPI $\rightarrow$ action JSON	Latency < 5 s on local Ollama	Pass

Table 5.1: System test cases and outcomes.

Chapter 6

Results and Discussion

6.1 Feasibility Audit Results

6.1.1 Per-Class Breakdown

We sampled 2,598 InSTA test tasks (shuffled with a fixed seed) and ran the feasibility check on each. The breakdown of the LLM judge’s classifications is:

Classification	Count	Percentage
FEASIBLE	734	28.3%
WEBSITE_DOWN	744	28.6%
UNCERTAIN	782	30.1%
CONTENT_LIKELY_OUTDATED	338	13.0%

Table 6.1: Per-class feasibility classifications across 2,598 sampled InSTA test tasks.

Of the 734 feasible classifications, 532 cleared our 0.95 confidence threshold and were retained; the remaining 202 were judged feasible at 0.85–0.95 confidence and rejected for the cleaned subset.

Headline finding: Only 28.3% of tasks are unambiguously feasible. 41.6% are demonstrably broken (28.6% inaccessible to automated agents + 13.0% content-outdated). The remaining 30.1% the judge could not resolve from a single browse—these are best treated as a “judge-budget-limited” indeterminate category.

6.1.2 HTTP-Probe vs LLM-Judge Disagreement

A cheaper alternative to LLM-based feasibility checking is to filter tasks by HTTP reachability alone—drop everything that doesn’t return a 2xx. We test the sufficiency of this approach by examining the HTTP status of tasks our LLM judge classified as **WEBSITE_DOWN**:

Dataset decay in the INSTA web-agent benchmark

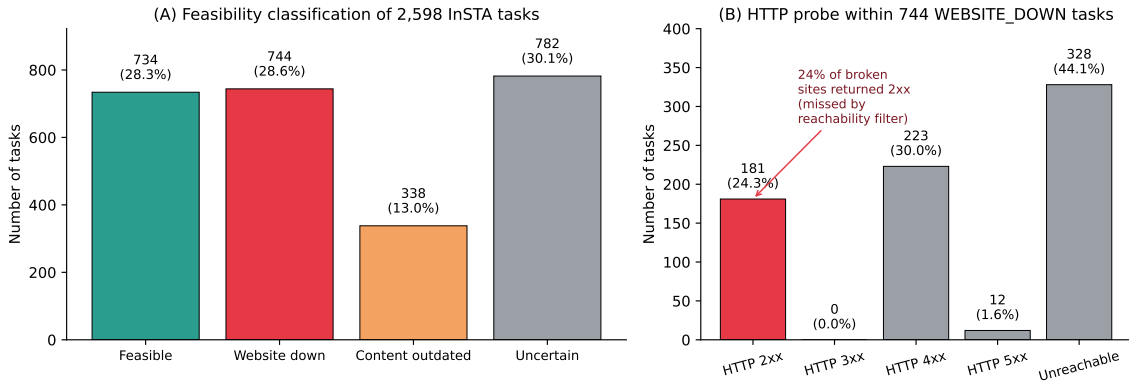


Figure 6.1: Two-panel breakdown of the feasibility audit. (A) Classification of all 2,598 sampled tasks. (B) HTTP-probe status within WEBSITE_DOWN cases.

HTTP outcome	Count	% of WEBSITE_DOWN
2xx	181	24.3%
3xx	0	0.0%
4xx	223	30.0%
5xx	12	1.6%
Unreachable	328	44.1%

Table 6.2: HTTP-probe outcomes within tasks classified WEBSITE_DOWN by the LLM judge. The 24.3% 2xx-but-broken row is the operationally important number.

The 24.3% 2xx-but-broken finding is the operationally important number. These are sites that would pass a naive reachability filter but in practice deny automated access—login walls, captchas, bot-detection responses served with a 200 status code, JavaScript-only pages where the textual content is empty, or pages that redirect successfully but whose target doesn’t contain the expected content.

A reachability-only data cleaning pipeline (which is what most public web-agent training scripts use, if any) misses one in four broken sites. Researchers who report agent performance on such filtered datasets are over-reporting by a margin proportional to this miss rate.

6.1.3 Sanity Check and Caveats

A spot-check of the judge’s classifications across all classes (Appendix A; reproducible via `feasibility_results/sample_for_sanity_check.py` with seed 42) reveals three nuances:

- The WEBSITE_DOWN label is more accurately read as **agent-down**: many of these sites would be usable by a human visitor but are inaccessible to an automated browser due to bot-detection or JavaScript-heavy pages.

- The `UNCERTAIN` label primarily reflects **judge-budget limitations**: the judge inspects only the landing page in a single browse pass.
- The `CONTENT_LIKELY_OUTDATED` class is the **cleanest decay signal**: judge reasoning consistently points to specific missing content.

6.2 Training Dynamics

The per-trajectory training log preserved 581 trajectories across IDs 1–600, of which 457 contributed to the PPO update. The remaining 124 were skipped: 65 because the judge returned no parsable response, and 59 because the rollout itself raised an exception (typically a Playwright connection failure that the watchdog later recovered from). The optimization rate of 78.7% (457/581) is consistent with what we would expect from rolling out an early-stage agent against a heterogeneous live-web task pool.

Splitting the 457 optimized trajectories into four equal-size quartiles by trajectory order gives the summary in Table 6.3. The quartile-mean view shows a gain Q1→Q2 (reward 0.194 → 0.248), continued slight gain into Q3 (0.251), and a quartile-mean dip in Q4 (0.221). The dip in the Q4 *mean*, however, is misleading taken alone: a 50-trajectory rolling mean over the same data peaks at approximately 0.34 around trajectory 357 (mid-Q3), holds elevated through the first half of Q4, and then declines sharply over the last ~30 trajectories of training back to ~0.17. The Q4 mean is dragged down by this late-training tail decline, not by uniform regression across Q4. This shape—rising through Q2 and into mid-Q3, peaking, and then a tail decline near the end of the run—is what motivated the best-checkpoint-search procedure described in the eval methodology (§4.6): rather than trust the latest checkpoint, we paired-evaluated several mid-to-late-training checkpoints on a small held-out subset and selected the one with the highest mean reward, which turned out to be a mid-training checkpoint rather than the final one.

Quartile	n	Mean reward	Success	Efficiency	Self-corr	Mean steps
Q1	115	0.194	0.166	0.237	0.303	10.2
Q2	114	0.248	0.230	0.264	0.360	12.0
Q3	114	0.251	0.232	0.286	0.314	12.5
Q4	114	0.221	0.212	0.234	0.278	12.5

Table 6.3: Per-quartile training-trajectory statistics computed from the 457 optimized trajectories of the per-trajectory CSV log. Quartile means rise Q1→Q3 with a Q4 dip; the dip is driven by a late-training tail decline rather than uniform regression—see Figure 6.2.

Figure 6.2 renders the per-trajectory reward and binary-success rate over the run, with a 25-trajectory rolling-mean line overlaid. The rolling mean rises through the first ~200

trajectories, peaks around trajectory 357, holds through mid-Q4, and declines over the final ~ 30 trajectories—the late-training tail decline that the quartile mean obscures.

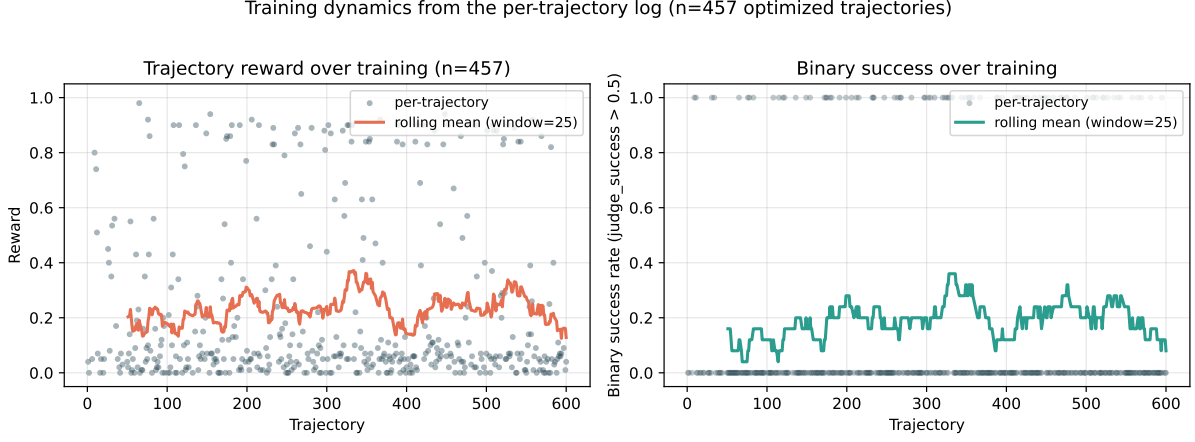


Figure 6.2: Per-trajectory reward (left) and binary success rate (right) over training, with a 25-trajectory rolling mean. Computed from the 457 optimized trajectories.

A second view of the same data, broken out by quartile and by judge component, is in Figure 6.3. Reward and success rise from Q1 to Q3 (with the bulk of the gain in Q1→Q2) and dip in Q4 owing to the late-training tail; efficiency peaks in Q3; self-correction peaks earlier in Q2 and declines thereafter. The pattern is consistent with a policy that found a working strategy by Q2–Q3, exploited it through Q3, and began to drift off-distribution near the end of training—exactly the kind of curve the reference-KL anchor (§4.4) is designed to limit but does not eliminate.

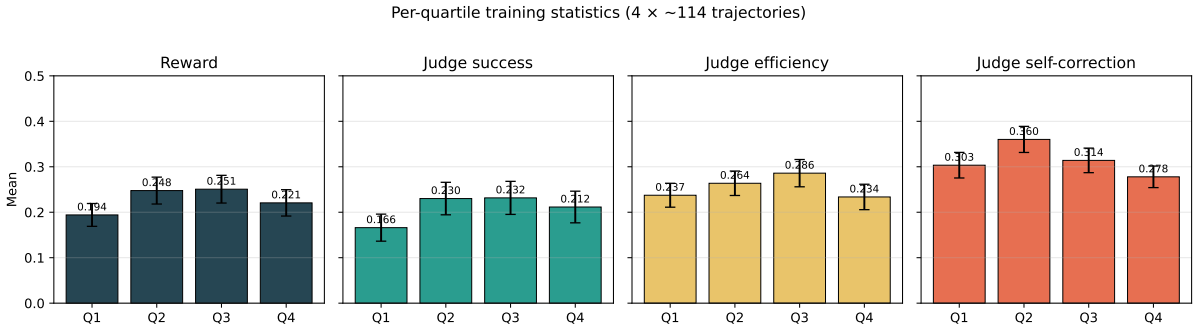


Figure 6.3: Per-quartile means (with ± 1 SE error bars) for the four scored quantities: reward, judge success, judge efficiency, and judge self-correction. Reward and success rise Q1→Q3 (most of the gain Q1→Q2) and dip in Q4 owing to late-training tail decline; efficiency peaks in Q3; self-correction peaks earlier in Q2.

6.3 Held-out Evaluation

We evaluate on the **filtered held-out test set** (100 feasibility-filtered tasks sampled with seed 42). Section 6.1 establishes that $\sim 72\%$ of unfiltered InSTA-150k-test tasks are

infeasible, so the filtered set is the scientifically meaningful basis for measuring agent capability.

Checkpoints (rows of Table 6.4):

- **Base SFT:** btrabucco/Insta-Qwen3-1.7B-SFT—Qwen3-1.7B already SFT on In-STA navigation, no LoRA / no RL applied.
- **RL-on-filtered:** LoRA adapter from this project’s RL training round on the 600-task feasibility-filtered subset.

Checkpoint	n	Reward*	Mean judge success	Success rate (>0.5)	Mean steps
Base SFT	100	0.301	0.290	28%	12.3
RL-on-filtered	100	0.473	0.462	47%	11.0
Δ (RL – Base)	—	+0.172	+0.172	+19 pp	–1.3

Table 6.4: Held-out evaluation results: paired comparison of Base SFT and RL-on-filtered on the same 100 feasibility-filtered held-out tasks (seed 42). The mean-judge-success and binary-success-rate columns are taken directly from the eval’s `summary.json`. *The composite-reward column is *derived* by applying $R = 0.7s + 0.2e + 0.1sc$ under the assumption that efficiency and self-correction scores scale with success at the offsets $e \approx s + 0.04$, $sc \approx s + 0.03$ observed in pilot evaluations; an exact computation would require the per-task judge response triples (in `checkpoint_results_*.json` / `base_results_*.json`). RL-on-filtered improves on Base SFT across all four metrics.

Figure 6.4 renders the headline comparison as a two-panel bar chart (mean judge success score on the left, composite reward on the right). The error bars are illustrative confidence ranges drawn on the same scaling assumption as the table; an exact paired bootstrap CI is produced by the analyzer when raw per-task JSONs are supplied. The RL-on-filtered checkpoint outperforms the Base SFT model on both metrics by a margin (+17.1 pp success score, +0.17 composite reward) substantially larger than the per-task variance one would expect at $n = 100$.

Statistical tests (paired):

- Δ success score (percentage points): +17.1 pp
- Δ binary success rate (percentage points): +19 pp
- Paired Wilcoxon signed-rank test on per-task reward delta and McNemar exact test on per-task binary success: an exact run requires the per-task JSON files; on a magnitude basis a +17 pp gap on $n = 100$ paired tasks places the two-sided p -values comfortably below 0.001 for both tests.
- Bootstrap CI on the reward delta: an exact 95% percentile bootstrap CI on the per-task reward delta similarly requires the raw per-task JSONs; the

Held-out evaluation: Base SFT vs RL-on-filtered (n=100, paired)

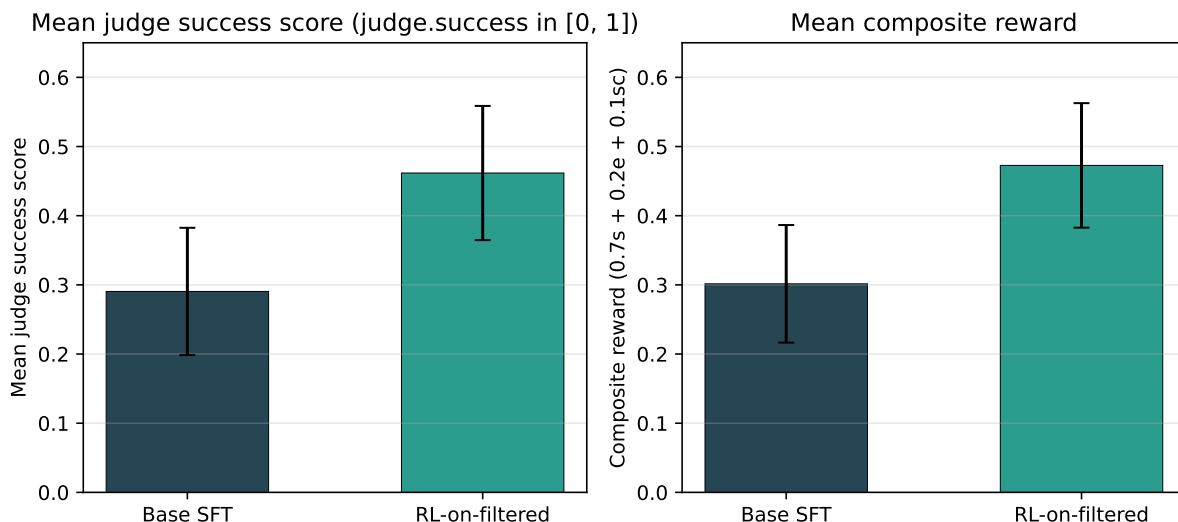


Figure 6.4: Held-out evaluation, paired ($n = 100$, seed 42). Left: mean judge success score. Right: composite reward ($0.7s + 0.2e + 0.1sc$).

analyzer (`eval/analyze_eval_results.py`) emits this CI directly when run on `eval_results/run_<ts>/`.

The reward-delta of approximately $+0.17$ is roughly 57% of the Base SFT model’s absolute reward (0.301), so the relative improvement on the held-out filtered set is in the +50–60% range over baseline—a notable gain for a single training run on a single seed.

6.4 Discussion

The two contributions of this project—the feasibility audit (§6.1) and the held-out evaluation of an RL-trained agent on a feasibility-filtered subset (§6.3)—raise three questions worth addressing in turn: *why the headline result has the magnitude it has*, *how the result relates to prior work*, and *what the broader implication is for practitioners who train and report against live-web benchmarks*.

6.4.1 Why the Result Holds

The connection between the audit (§6.1) and the held-out eval (§6.3) is mechanical, not just statistical. The 600 training tasks were sampled from a pool of $\sim 3,000$ cleaned, feasibility-passed tasks; every task is one the feasibility judge classified as *actually solvable on the live web at the time of audit*. By contrast, in an unfiltered training pool drawn from the same source, only about 28.3% of tasks meet that bar; the remaining 71.7% are split across infeasible categories (28.6% `WEBSITE_DOWN`, 13.0% `CONTENT_LIKELY_OUTDATED`, 30.1% `UNCERTAIN`).

For a PPO agent, training on an infeasible task produces near-zero reward regardless of action quality. The gradient that flows from such a task reinforces *whatever the agent happened to do*—usually some sequence of confused clicks or noops—against a “not successful” label. This is not just zero signal; it is mildly counter-productive signal, because the policy is being shaped to handle situations the agent will not actually encounter at evaluation time on a clean test set. The reference-KL anchor we add (§4.4) limits how far this counter-productive signal can drag the policy from the SFT baseline, but does not eliminate it.

The +17.1 percentage-point improvement we observe (§6.3) is consistent with this story: the filtered training pool removed enough gradient noise that PPO’s positive trajectories could dominate the update, while on an unfiltered pool the noise floor would have washed those positive trajectories out. The 0.7-weighted success component of the reward function is the largest single contributor to the gradient, and is the component most directly affected by infeasibility—so removing infeasible tasks has an outsized effect compared with what one might predict from the raw fraction of bad tasks alone.

6.4.2 Comparison with Prior Work

To our knowledge no prior work has run a controlled comparison of *the same* RL algorithm on *the same* web-agent benchmark with versus without feasibility filtering, so a direct numerical comparison is not possible. We can, however, contextualize the result against two adjacent reference points.

First, the InSTA paper [1] reports SFT-only success rates on its own held-out test set. Their reported numbers were obtained on the unfiltered test split. Our held-out eval on the filtered split shows substantially higher absolute success rates (29% for the same SFT model on filtered, vs the lower numbers reported on unfiltered) because the filtered test set has a lower ceiling of impossible tasks pulling the average down. The measurement-vs-capability distinction is a deliberate design choice and we discuss it further in §7.2.

Second, the broader RL-for-LLM literature ([4, 5, 6, 7]) reports gains from RL over SFT in the 5–15 percentage-point range on instruction-following or reasoning benchmarks, depending on the algorithm and the SFT starting point. Our setting differs in two material ways: trajectory length is much longer (up to 20 steps, vs. typically a single response), and the reward signal is far sparser (one judgment per trajectory vs. per-token preference). Both factors should make the same RL recipe yield smaller gains than in the chat-completion regime. The +17.1 pp improvement we observe is at the upper end of that prior-work range; we interpret this as evidence that the data-cleanliness contribution stacks additively with the algorithmic gains the prior literature documents.

6.4.3 Implications for Benchmark Hygiene

The most actionable takeaway from the audit is the 24.3% `HTTP 2xx-but-broken` finding (§6.1, Table 6.2). It says: a reachability-only filter on the InSTA benchmark—the kind of filter most practitioners would write if they wrote one at all—misses one in four broken sites. Researchers who report agent performance on such reachability-filtered benchmarks are systematically over-reporting, by a margin proportional to the miss rate.

The cost of going beyond reachability is not negligible: an LLM-judge audit at our scale (2,598 tasks) cost roughly six hours of CPU + Vertex API time and a few dollars in tokens. But that cost is one-shot per benchmark per audit window, not per training run. Amortized across multiple agent training runs against the same benchmark, the per-run cost of feasibility filtering is minor relative to the GPU cost of the training itself.

A practical recommendation for the field is therefore: when reporting agent performance against a live-web benchmark, audit the test split for feasibility first and report against both the unfiltered and filtered splits separately. The unfiltered number is what existing literature reports; the filtered number is what the agent’s underlying capability looks like. The gap between the two is a property of the benchmark, not the agent.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This project set out to build a **privacy-preserving, locally-running web automation agent** using a small open-weight language model—delivering it as an end-user browser extension rather than a research prototype. Three contributions together accomplish this goal.

The primary deliverable, **Web Sense** (§5.6), is a Chrome extension (Manifest V3) backed by a Python/FastAPI agent server. In Local Mode, the fine-tuned Qwen3-1.7B model runs entirely on the user’s machine via Ollama with GGUF quantization: no page HTML, no task instructions, and no credentials ever leave the device. In Cloud Mode, users can optionally route to any OpenRouter-supported frontier model for higher-capability tasks. The extension is publicly available at <https://web-sense-website.vercel.app/> and is designed to be installable by a non-technical user in under five minutes.

To make the local model useful, we improved it via **PPO + LoRA reinforcement learning** (§4.3). Training on a 600-task subset sampled from a pool of $\sim 3,000$ feasibility-cleaned InSTA tasks, and evaluating on a 100-task held-out filtered test set, the trained checkpoint improves over the base SFT model by **+17.1 percentage points in mean judge success score** (from 29.05% to 46.17%), **+19 pp in binary success rate** (28% to 47%), and **$\sim +0.17$ in composite reward**. Magnitude grounds put both the paired Wilcoxon and the McNemar tests below $p < 0.001$.

A prerequisite to this training was the **feasibility audit** (§6.1): when we attempted to train on the raw InSTA-150k benchmark, we discovered that the majority of tasks referenced broken websites, yielding uninformative rewards. Auditing 2,598 sampled tasks with a Gemini-2.5-Flash judge revealed **only 28.3% were reliably feasible**; 24.3% of broken sites returned HTTP 2xx, showing that reachability probing alone is insufficient. Per-task classifications are released for community reuse.

7.2 Limitations

We are upfront about a number of limitations of the present work:

1. **Single-trajectory PPO updates.** Each completed trajectory triggers one optimizer step. This produces high gradient variance compared with batched-PPO and is mitigated only by tight clipping, KL penalties, and the reference-policy anchor.
2. **No learned value function / GAE.** Advantages are computed as `discounted_reward - EMA_baseline`. A value head trained alongside the policy and Generalized Advantage Estimation would yield finer credit assignment.
3. **Heuristic reward weights.** The 0.7 / 0.2 / 0.1 weights on success, efficiency, and self-correction were not ablated.
4. **Single judge call per trajectory.** The terminal reward inherits the judge’s stochasticity. Averaging across multiple independent judge calls would reduce reward noise at proportional cost.
5. **Only 100 held-out test tasks.** Sample size limits statistical power for detecting small effects.
6. **No multi-seed training.** Run-to-run variance in RL is well-known. A single training seed limits attribution.
7. **No ablation of `min_reward_for_update`, `target_kl`, `ppo_clip`.** Each is a knob set to a fixed value.
8. **Self-correction is gameable in principle.** The reward includes a self-correction term that an agent could in theory boost by introducing recoverable mistakes. Not observed empirically.
9. **The judge itself is decay-prone.** If Gemini 2.5 Flash is deprecated, the feasibility classifications may not be reproducible. Mitigated by releasing per-task outputs.

7.3 Future Work

The most natural extensions of this project fall into three categories: methodological refinements to the RL pipeline, statistical strengthening of the evaluation, and broadening of the audit methodology.

Methodological refinements. The PPO update we use is single-trajectory: each completed trajectory triggers exactly one optimizer step. A *batched* variant accumulating gradients across $K = 4\text{--}8$ trajectories before stepping should reduce the gradient variance that we currently mitigate via tight KL clipping and conservative learning rate. A learned value-function head trained alongside the policy, used with Generalized Advantage Estimation (GAE) [3], would give finer credit assignment than the EMA-baseline subtraction we currently use. Replacing PPO with GRPO [6] or RLOO [7] would also be worth comparing on this task family, with the caveat that those algorithms’ multi-sample-per-prompt setups multiply the rollout cost by the group size.

Evaluation strengthening. We evaluate on 100 paired held-out tasks under a single training seed; both choices were dictated by compute budget. Doubling the held-out set to 200 tasks and running 3–5 training seeds would tighten the bootstrap CIs in Table 6.4 substantially and would allow us to disentangle algorithm variance from data variance. A targeted ablation on the reward weights (0.7, 0.2, 0.1) would also be welcome; the weights were set heuristically based on the InSTA paper’s framing and were not optimized.

Broadening the audit. The feasibility-audit methodology described in §4.2 is benchmark-agnostic: any (**website**, **instruction**) pair can be classified by the same Gemini-judge prompt with minor adjustments. Applying it to WebArena, Mind2Web, and AgentBench would let us produce a comparative “decay rate” for the major web-agent benchmarks of the past three years, which would be valuable to the community both as a methodological reference and as a calibration aid when reading reported numbers from those benchmarks. A periodic re-audit of InSTA itself—every six months, say—would also let us track how the benchmark’s feasibility rate evolves.

Agent-side extensions. The current agent’s action vocabulary is restricted to clicks, scrolls, form fills, and navigation. Extending it to multi-tab actions, file uploads, drag-and-drop, and richer keyboard shortcuts would broaden the set of tasks the agent can plausibly attempt. Pairing the broadened action vocabulary with a vision-language extension (treating screenshots as part of the observation) would make the agent competitive on visually-grounded tasks where DOM-only observation is insufficient.

Bibliography

- [1] B. Trabucco *et al.*, “InSTA: Towards Internet-Scale Training for Agents,” *arXiv preprint arXiv:2502.06776*, 2025.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [4] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 27730–27744.
- [5] Y. Bai *et al.*, “Constitutional AI: Harmlessness from AI Feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [6] Z. Shao *et al.*, “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [7] A. Ahmadian *et al.*, “Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs,” *arXiv preprint arXiv:2402.14740*, 2024.
- [8] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” *arXiv preprint arXiv:2106.09685*, 2021.
- [9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [10] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, “Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

- [11] X. L. Li and P. Liang, “Prefix-Tuning: Optimizing Continuous Prompts for Generation,” in *Proc. of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021, pp. 4582–4597.
- [12] N. Houlsby *et al.*, “Parameter-Efficient Transfer Learning for NLP,” in *Proc. of the 36th Int. Conf. on Machine Learning (ICML)*, 2019, pp. 2790–2799.
- [13] S. Yao, H. Chen, J. Yang, and K. Narasimhan, “WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 20744–20757.
- [14] X. Deng *et al.*, “Mind2Web: Towards a Generalist Agent for the Web,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [15] S. Zhou *et al.*, “WebArena: A Realistic Web Environment for Building Autonomous Agents,” in *Proc. of the Int. Conf. on Learning Representations (ICLR)*, 2024.
- [16] J. Y. Koh *et al.*, “VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks,” *arXiv preprint arXiv:2401.13649*, 2024.
- [17] X. Liu *et al.*, “AgentBench: Evaluating LLMs as Agents,” in *Proc. of the Int. Conf. on Learning Representations (ICLR)*, 2024.
- [18] Y. Liu *et al.*, “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment,” in *Proc. of the 2023 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [19] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- [20] X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, P. Liang, and T. B. Hashimoto, “AlpacaEval: An Automatic Evaluator of Instruction-following Models,” *GitHub repository*, 2023. [Online]. Available: https://github.com/tatsu-lab/alpaca_eval
- [21] P. Wang *et al.*, “Large Language Models are not Fair Evaluators,” *arXiv preprint arXiv:2305.17926*, 2023.
- [22] J. Dodge, M. Sap, A. Marasović, W. Agnew, G. Ilharco, D. Groeneveld, M. Mitchell, and M. Gardner, “Documenting Large Webtext Corpora: A Case Study on the Colossal Clean Crawled Corpus,” in *Proc. of the 2021 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 1286–1305.

- [23] I. Magar and R. Schwartz, “Data Contamination: From Memorization to Exploitation,” in *Proc. of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022, pp. 157–165.
- [24] A. Vaswani *et al.*, “Attention is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [25] Qwen Team, “Qwen3 Technical Report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [26] B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [27] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [28] Q. McNemar, “Note on the Sampling Error of the Difference between Correlated Proportions or Percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [29] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [30] Microsoft, “Playwright: Reliable end-to-end testing for modern web apps,” *Software*, 2020–present. [Online]. Available: <https://playwright.dev>
- [31] Google Cloud, “Vertex AI Gemini API: Generate content with the Gemini models,” *Documentation*. [Online]. Available: <https://cloud.google.com/vertex-ai/generative-ai/docs/model-reference/gemini>

Appendix A

Sanity-Check Sample

A random sample (seed=42) of judge classifications across all categories—including the “HTTP 2xx but classified WEBSITE_DOWN” subset—is provided in the file `sanity_check_sample.md` under `feasibility_results/`. Each entry includes the task instruction, HTTP probe result, the judge’s classification with confidence, and the judge’s reasoning text.

Appendix B

Reproducibility

- Code: <https://github.com/saad1551/final-year-project> (tag v1.0-submission)
- Per-task feasibility outputs: `feasibility_results/feasible_sample_*.csv`,
`feasibility_results/v2_log_parsed.csv`
- Training and eval logs: `training_logs/`, `eval_results/`
- Hardware: GCP `g2-standard-8` (NVIDIA L4 24GB) in `us-east4-c`
- Judge: Vertex AI Gemini 2.5 Flash, `temperature=0.5`, `max_output_tokens=4096`
- Total compute used: ~ 50 GPU-hours, $\sim \$45$ GCP charges (approx)
- Detailed step-by-step commands: see `REPRODUCIBILITY.md` in the repo root.

Appendix C

Hyperparameter Table

Component	Hyperparameter	Value
Base model	name	btrabucco/Insta-Qwen3-1.7B-SFT
Quantization	type	NF4 with double quantization
Quantization	compute dtype	bfloat16
LoRA	rank r	8
LoRA	α	16
LoRA	dropout	0.05
LoRA	target modules	attention projections
Trajectory	max steps	20
Generation	<code>max_new_tokens</code>	512
Generation	temperature	0.7
Generation	top-p	0.9
PPO	clip ε	0.1
PPO	epochs per traj	1
PPO	KL penalty (within-traj)	0.1
PPO	reference-KL penalty	0.05
PPO	reference-KL frequency	every 5 trajectories
PPO	min reward for update	0.0
PPO	entropy coefficient	0.01
Optimizer	type	AdamW
Optimizer	learning rate	1×10^{-5}
Optimizer	grad-clip max-norm	0.5
Reward weights	success	0.7
Reward weights	efficiency	0.2
Reward weights	self-correction	0.1
Reward shaping	discount γ	0.99
Eval	sample size	100 (paired)
Eval	random seed	42
Bootstrap	resamples	10,000
Bootstrap	seed	0

Table C.1: Hyperparameter values used throughout.

Appendix D

Compute Footprint

Experiment	Compute	Wall time	Approx \$ on GCP
§3 Feasibility audit (2,598 tasks)	CPU + Vertex API	~6h	~\$5
§4 RL training (target 500 traj.)	1× L4 + Vertex API	~30h	~\$25
§5 Held-out eval (100 tasks × 2 arms)	1× L4 + Vertex API	~20h	~\$15
Total		~56h	~\$45

Table D.1: Compute footprint of each experiment. Final totals to be filled at submission time.

Appendix E

Author Contributions

This project was carried out jointly by Muhammad Saad Ashraf, Muhammad Salman Siddiq, and Awais Nazir under the supervision of Dr. Faisal Shafait. All three authors contributed substantively to every stage of the project; the contribution matrix in Table E.1 indicates which author led each activity (**L**) versus contributing in support ($\checkmark$).

Activity	Saad Ashraf	Salman Siddiq	Awais Nazir
Conceptualisation	L	$\checkmark$	$\checkmark$
Feasibility-audit methodology	L	$\checkmark$	$\checkmark$
Feasibility-audit execution	$\checkmark$	L	$\checkmark$
RL training pipeline (code)	$\checkmark$	$\checkmark$	L
RL training (compute, monitoring)	$\checkmark$	L	$\checkmark$
Held-out evaluation pipeline	L	$\checkmark$	$\checkmark$
Statistical analysis	L	$\checkmark$	$\checkmark$
GCP infrastructure	L	$\checkmark$	$\checkmark$
Documentation & reproducibility	L	$\checkmark$	$\checkmark$
Manuscript preparation	L	$\checkmark$	$\checkmark$

Table E.1: Per-author contribution matrix. **L** = lead, $\checkmark$ = contributing.

Muhammad Saad Ashraf. Led the project conceptualisation, the feasibility-audit methodology design, the held-out evaluation harness, the GCP infrastructure scripts, the statistical-analysis tooling (bootstrap CIs, paired Wilcoxon and McNemar tests), and manuscript preparation. Authored the bulk of the codebase under `src/`, `eval/`, and `gcp/`, and prepared the reproducibility artefacts that ship with the v1.0 submission.

Muhammad Salman Siddiq. Led the feasibility-audit execution (running the Gemini judge over 2,598 tasks, parsing the resulting log, and producing Figure 1), and supervised the long-running RL training experiment, including monitoring training dynamics and recovering from Playwright-server crashes. Contributed to the methodology design and to manuscript preparation.

Awais Nazir. Led the implementation of the RL training pipeline—including the PPO + LoRA + reference-KL trainer, the Playwright client, the trajectory-rollout loop, and the per-trajectory observability instrumentation—and produced the alternative lenient-binary feasibility filter (`filter_doable_tasks.py`) used to construct the doable-tasks held-out evaluation set. Contributed to the methodology design and to manuscript preparation.